A

Project Report

on

BrakeGuard-XAI – An Advanced Secure Explainable AI Paradigm for Early-Stage Brake Anomaly Detection and Interpretable Predictive Maintenance

Submitted in partial fulfillment of the requirement

For the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

GOKEDA VEERA SATYA SRI PRAVALLIKA

22A31A0508

Under the esteemed guidance of

Dr. Y. Jayababu
Professor
Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PRAGATI ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, KAKINADA, Accredited by NBA)

1-378, ADB Road, Surampalem, Near Peddapuram-533437

2025-26

PRAGATI ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada, Accredited by NBA)

1-378, ADB Road, Surampalem, Near Peddapuram-533437

CERTIFICATE

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Learning is Supreme Deity

This is to certify that the project report entitled **"BrakeGuard-XAI – An Advanced Secure Explainable AI Paradigm for Early-Stage Brake Anomaly Detection and Interpretable Predictive Maintenance"** is being submitted by **GOKEDA VEERA SATYA SRI PRAVALLIKA (22A31A0508),** in partial fulfillment for the award of the Degree of **Bachelor of Technology**, during the year **2025-26** in Computer Science and Engineering of Pragati Engineering College, for the record of a bonafide work carried out by them.

Project Guide:

Dr.Y.Jayababu Ph.D
Professor
Department of CSE

Head of the Department:

Dr.D.V.Manjula Ph.D,PDF
Associate Professor & HoD
Department of CSE

ACKNOWLEDGEMENT

We express our thanks to project guide **Dr. Y. Jayababu, Professor** of **Computer Science Engineering**, who deserves a special note of thanks and gratitude, for having extended their fullest co-operation and guidance, without this, project would never have materialized.

We express our deep sense of gratitude to **Dr. D. V. Manjula, Associate Professor and Head of the Department** of Computer Science Engineering, for having shown keen interest at every stage of development of our project and for guiding us in every aspect.

We wish to express our special thanks to our beloved **Dr. K. SATYANARAYANA, Professor & Academic Director** for giving guidelines and encouragement.

We wish to express our special thanks to our beloved **Dr. G. NARESH, Professor & Principal** for giving guidelines and encouragement.

We wish to express sincere gratitude to our beloved and respected **Dr. P. KRISHNA RAO, Chairman** and **Sri. M. V. HARANATHA BABU, Director (Management)** and **Sri. M. SATISH, Vice-President** for their encouragement and blessings.

We are thankful to all our faculty members of the Department for their valuable suggestions. Our sincere thanks are also extended to all the teaching and non-teaching staff of Pragati Engineering College.

We are grateful to our parents and family members for the continuous encouragement throughout the completion of our project.

GOKEDA VEERA SATYA SRI PRAVALLIKA

22A31A0508

ABSTRACT

Brake system failure is a major safety concern in heavy transport vehicles, as it can lead to accidents, high maintenance costs, and operational delays. Air brake systems used in heavy vehicles are complex and generate large amounts of sensor data during operation. Traditional maintenance methods are reactive and may fail to detect faults at an early stage. To overcome these limitations, predictive maintenance using machine learning has become an effective solution. However, many machine learning models lack transparency, making their decisions difficult to understand and trust.

This project proposes a secure and explainable artificial intelligence (AI) approach for predicting brake faults in heavy transport vehicles. Sensor data from the air pressure system are analyzed using multiple machine learning classification techniques to identify potential brake failures. Random Forest is selected as the primary model due to its high prediction accuracy. To improve transparency and trust, Explainable AI techniques such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME) are used to explain feature contributions in model predictions. Important features are identified, and model complexity is reduced without affecting performance. The proposed approach enhances fault prediction accuracy, improves interpretability, and supports efficient and reliable predictive maintenance for heavy transport systems.

TABLE OF CONTENT

ACKNOWLEDGEMENT

ABSTRACT

LIST OF FIGURES

CHAPTER-1 Introduction ----------1-4

CHAPTER-2 Literature Review---------- 5-7

CHAPTER -3 System Analysis----------8-13

3.1 Existing System

3.1.1 Disadvantage of Existing System

3.2 Proposed System

3.2.1 Advantages of proposed System

3.3 System Requirements Specification

CHAPTER-4 System Design ---------- 14-21

4.1 System Architecture

4.2 Data Flow Diagram

4.3 Flow Chart Diagram

4.4 Activity Diagram

4.5 Class Diagram

4.6 Use Case Diagram

4.7 Sequence Diagram

CHAPTER-5 Implementation ---------- 22-42

CHAPTER-6 Testing----------43-48

CHAPTER-7 Results ---------- 49-57

CHAPTER-8 Conclusion and Future Enhancement ---------- 58-60

REFERENCES ---------- 61-62

PAPER PUBLICATION ---------- 63

LIST OF FIGURES

Figure No.	Name of the figure	Page No.
4.1	System Architecture	15
4.2	Data Flow Diagram	16
4.3	Remote User Flow Chart	17
4.4	Activity Diagram	18
4.5	Class diagram	19
4.6	Use - Case diagram	20
4.7	Sequence diagram	21
5.1	Django Architecture Diagram	26
5.2	Django Working Flow Diagram	26
7.1	Home Screen	50
7.2	Remote User Login Page	50
7.3	Admin login page	51
7.4	User Registration Form	51
7.5	View Remote user profile	52
7.6	Predicting Software Defect	52
7.7	Admin dashboard	53
7.8	User dashboard	53
7.9	User feedback form	53
7.10	Random forest model execution dashboard	54
7.11	Decision tree model execution dashboard	54
7.12	Logistic model execution dashboard	55
7.13	GBoost model execution dashboard	55
7.14	Model accuracy graph	56
7.15	Feedback analysis	56
7.16	Contact us form	57
7.17	About Us	57

LIST OF TABLES

Table No. **Name of the Table** **Page No.**

6.1 UNIT TEST CASES ---------- 46-47

6.1.1 Data Upload Module

6.1.2 Data Preprocessing Module

6.1.3 Model Training Module

6.1.4 Prediction Module

6.1.5 Explainability Module (XAI)

6.1.6 Interface Module

6.2 Black Box Test Cases ---------- 47-48

6.2.1 Boundary Value Testing

6.2.2 Functional Testing (Black Box)

6.2.3 Error Handling Testing

6.2.4 Performance Testing

6.2.5 Security Testing

CHAPTER-1
INTRODUCTION

CHAPTER-1

INTRODUCTION

In modern transportation systems, safety and reliability are critical factors, especially in heavy commercial vehicles where braking systems play a vital role in preventing accidents. Brake system failure is a serious issue that can lead to severe consequences such as loss of life, damage to goods, increased maintenance costs, and operational delays. Heavy transport vehicles, including trucks and buses, primarily depend on air brake systems due to their efficiency and reliability. However, these systems are complex in nature, consisting of multiple interconnected components such as compressors, air reservoirs, valves, sensors, and brake chambers. Over time, these components are vulnerable to faults caused by wear and tear, pressure leakage, environmental conditions, and sensor failures.

Safety and reliability are very important in heavy transport vehicles, especially in braking systems, as brake failures can lead to serious accidents, loss of life, increased maintenance costs, and operational delays. Heavy commercial vehicles mainly use air brake systems because of their efficiency, durability, and ability to handle high loads. However, these systems are complex in nature and may develop faults over time due to factors such as pressure leakage, component wear, sensor malfunction, and environmental conditions. These faults, if not detected early, can reduce braking efficiency and increase the risk of vehicle failure.

Traditional maintenance approaches used in heavy transport systems are mostly reactive or schedule-based. In reactive maintenance, faults are identified only after failure occurs, while in scheduled maintenance, inspections are performed at regular intervals regardless of system condition. Both approaches have limitations, as they either increase downtime and repair costs or fail to detect hidden faults at an early stage. Therefore, there is a strong need for an intelligent system that can predict brake faults in advance and support preventive maintenance. With the rapid advancement of technology, modern vehicles are equipped with various sensors that continuously monitor system parameters such as air pressure, temperature, speed, and mechanical conditions.

Machine Learning (ML) has emerged as a powerful tool for analyzing such data and identifying patterns that indicate potential system failures. By using ML models, it is possible to predict faults before they occur, thereby improving safety, reducing maintenance costs, and increasing system reliability.

Although machine learning models provide high accuracy in prediction, many of them operate as “black-box” systems. This means that the internal decision-making process of these models is not easily understandable by humans. In safety-critical applications such as brake systems, it is important for engineers and maintenance personnel to understand why a particular prediction is made. Lack of transparency reduces trust in the system and limits its practical implementation.

To address this issue, Explainable Artificial Intelligence (XAI) has been introduced. XAI focuses on making machine learning models more transparent and interpretable by providing explanations for their predictions. Techniques such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) help in identifying the contribution of each feature to the final prediction. These methods allow users to understand how different sensor parameters influence the detection of brake faults, thereby increasing trust and usability of the system.

In addition to explainability, security and data privacy are also critical aspects when dealing with industrial and vehicle sensor data. Since the data is sensitive and often used in real-world operations, it must be protected from unauthorized access and misuse. A secure system ensures data integrity and reliability while maintaining the accuracy of predictions. Therefore, combining security with explainability and predictive modeling is essential for building a robust and trustworthy system.

This project presents a Secure Explainable-AI Approach for Brake Fault Prediction in Heavy Transport Vehicles. The system uses sensor data collected from the air pressure system of heavy vehicles and applies machine learning algorithms to detect potential brake faults. Among various models, the Random Forest algorithm is selected due to its high accuracy, robustness, and ability to handle large and complex datasets. To improve interpretability, XAI techniques such as SHAP and LIME are integrated into the system to explain model predictions and identify important features.

The proposed system also includes data preprocessing techniques such as handling missing values and balancing the dataset to improve model performance. Feature selection is performed to reduce model complexity and computational cost without affecting accuracy. By combining predictive accuracy, transparency, and security, the system provides an effective solution for early fault detection and supports better maintenance decision-making.

Overall, the proposed approach enhances vehicle safety, reduces unexpected failures, and improves maintenance efficiency. It also builds confidence in AI-based systems by providing clear explanations for predictions, making it suitable for real-world applications in heavy transport systems.

A typical feasibility study examines three core areas:

1. **Economic Feasibility**
2. **Technical Feasibility**
3. **Social Feasibility**

1. **Economic Feasibility :**

   Economic feasibility evaluates the cost-effectiveness of the system. The project uses open-source tools, reducing software expenses. Minimal hardware requirements further lower costs. It helps in reducing maintenance and repair expenses. Early fault detection prevents major failures and saves money. Hence, the system is financially beneficial.

2. **Technical Feasibility :**

   Technical feasibility checks whether the system can be implemented with available resources. The project uses standard technologies like Python and machine learning algorithms. It does not require advanced hardware or infrastructure. Data preprocessing and efficient models ensure smooth performance. The system can be easily deployed and maintained. Therefore, it is technically feasible.

3. **Social Feasibility :**

   Social feasibility focuses on user acceptance and usability. The system provides explainable results using SHAP and LIME. This improves trust among engineers and technicians. It is easy to use with proper training. Hence, it is socially acceptable and beneficial.

CHAPTER-2

LITERATURE REVIEW

CHAPTER-2

LITERATURE REVIEW

This study focuses on predicting brake faults in heavy transport vehicles using machine learning techniques applied to sensor data from air pressure systems. Various classification algorithms were evaluated, and the Random Forest model achieved the highest accuracy of 99.4%, demonstrating its effectiveness in handling large-scale sensor data and improving predictive performance [1]. Balancing explainability with privacy and security is a key challenge in modern AI systems, especially in industrial and transportation domains where sensitive data is involved. Research emphasizes the need to develop AI models that are both transparent and secure to ensure trust and compliance with regulatory requirements [2].

Machine learning-based fault detection methods have been widely applied in vehicle systems. Raveendran et al. implemented Decision Tree and Random Forest algorithms to analyze sensor data and identify faults, demonstrating that Random Forest performs better due to its robustness, ability to reduce overfitting, and capability to handle high-dimensional data efficiently [3]. Studies on braking dynamics and system modeling provide deeper insights into the behavior of heavy transport vehicles, helping improve the accuracy of fault detection and prediction systems [4].

These models contribute to understanding the relationship between pressure variations and braking efficiency. Explainable AI techniques have been increasingly applied in predictive maintenance systems to improve interpretability and user trust. However, challenges such as handling imbalanced datasets, missing values, and noisy sensor data still affect the performance and reliability of machine learning models [5].The SHAP method provides a unified framework for interpreting model predictions by assigning importance values to each feature, enabling users to understand how input variables influence the output [6].

Further research on Explainable AI explores its concepts, taxonomies, and challenges, highlighting the importance of transparency, accountability, and fairness in AI-based decision-making systems [7]. The integration of Artificial Intelligence with Internet of Things (IoT) technologies enables smart monitoring systems that improve automation, real-time data processing & system efficiency in various applications [8].

Machine learning algorithms such as Support Vector Machines (SVM) have also been successfully applied in different domains, demonstrating the versatility of AI techniques in classification and prediction tasks [9]. AI technologies have been explored in financial and regulatory systems, showing their capability to support complex decision-making processes and improve system performance [10].

Intrusion detection systems using artificial intelligence enhance system security by identifying abnormal patterns and preventing potential cyber threats in intelligent vehicle and communication systems [11]. The LIME (Local Interpretable Model-Agnostic Explanations) technique provides local explanations for individual predictions, helping users understand the reasoning behind model outputs and improving trust in AI systems [12].

Explainability in AI systems is essential to meet legal and regulatory requirements, ensuring that automated decisions are transparent, accountable, and understandable to users [13]. Privacy-preserving mechanisms combined with explainability improve data protection while maintaining model transparency, which is crucial in sensitive applications such as transportation and healthcare [14].

Modeling and simulation of brake systems enable real-time estimation of pressure and detection of faults, contributing to improved system monitoring and maintenance strategies [15]. Studies on vehicle brake system failures provide insights into common issues, maintenance practices, and repair strategies, which are essential for improving system reliability and safety [16].

Advancements in brake-by-wire technology enhance braking control systems, improving response time, safety, and overall performance of modern vehicles [17]. Predictive maintenance using machine learning techniques improves operational efficiency, reduces downtime, and supports sustainable industrial systems by enabling early fault detection [18].

Trust in AI systems is a critical factor, particularly in safety-critical applications such as transportation, where users must rely on accurate and explainable predictions for decision-making [19]. Advanced technologies such as digital twins and smart systems enhance system performance by integrating real-time data, simulation, and intelligent analytics, leading to improved efficiency, security, and reliability [20].

CHAPTER-3

SYSTEM ANALYSIS

CHAPTER-3

SYSTEM ANALYSIS

3.1 Existing System

The existing system for brake fault detection in heavy transport vehicles is based on traditional maintenance methods combined with basic data analysis and machine learning techniques. The system primarily follows reactive and scheduled maintenance approaches, where inspections are carried out either after a failure occurs or at regular time intervals.

In this system, technicians perform manual inspections to examine key components of the brake system such as air pressure units, brake chambers, valves, and pipelines. These inspections help ensure that the braking system is functioning properly. Along with manual methods, modern vehicles are equipped with sensors that continuously collect data related to parameters like pressure, temperature, and overall mechanical performance.

The collected sensor data is processed and analyzed to understand the behavior of the brake system. Basic data analysis techniques and classification methods are applied to determine whether the system is in a normal or faulty condition. Some existing systems also incorporate machine learning algorithms such as Decision Tree, Random Forest, and Support Vector Machine to analyze historical and real-time data for fault detection.

Additionally, predictive maintenance techniques are used in certain systems, where historical data is utilized to predict possible failures in advance. Overall, the existing system integrates traditional inspection practices with sensor-based data collection and machine learning methods to monitor and detect brake faults in heavy transport vehicles.

Modules of Existing System

1. **Data Collection Module**
2. **Data Preprocessing Module**
3. **Feature Extraction Module**

4. **Data Analysis Module**
5. **Machine Learning Module**
6. **Fault Detection Module**

The existing system also plays an important role in improving the reliability of heavy transport vehicles by continuously monitoring the performance of the braking system. It helps in identifying variations in sensor data and ensures that the system operates within safe limits. By combining sensor-based monitoring with analytical techniques, the system provides useful information to technicians for better understanding of brake conditions. This approach supports efficient maintenance practices and contributes to smoother vehicle operation.

3.1.1 Disadvantages of Existing System

Despite advancements in brake fault prediction techniques, several challenges still exist:

1. **Limited Interpretability of Complex Models:**

   Although Explainable AI techniques like SHAP and LIME are used, the core model (Random Forest) can still act like a black box. The explanations generated may not always be simple or easy to understand. Non-technical users may find it difficult to interpret feature contributions. Complex interactions between features can make explanations less clear. This may reduce trust in critical situations.

2. **Dependence on Data Quality:**

   The system strongly depends on the quality of input sensor data. If the data contains noise, missing values, or incorrect readings, the predictions may be affected. Poor-quality data can lead to inaccurate fault detection. Sensor failures or inconsistencies also reduce model reliability. Proper data cleaning and validation are essential.

3. **High Computational Cost:**

   Training multiple machine learning models requires significant computational resources. Explainability techniques like SHAP add extra processing time. Large datasets further increase memory and time requirements. This can make the system slower and costly to deploy. Efficient optimization is needed to reduce complexity.

4. **Limited Real-Time Capability:**

   The system mainly works on pre-collected datasets. It does not fully support real-time data processing. Without IoT integration, instant fault detection is not possible. This limits its use in live vehicle monitoring.

5. **Generalization Issues:**
   The model is trained on a specific dataset and environment. It may not perform well on different vehicle types or conditions. Variations in sensor behavior can affect accuracy. The system needs retraining for new datasets. Proper validation is required for wider usage.
6. **Security Challenges:**
   The system deals with sensitive vehicle sensor data. Protecting this data from unauthorized access is challenging. Data transmission and storage must be secured properly. Any security breach can affect system reliability. Strong encryption and security measures are required.
7. **Model Maintenance and Updates:**
   The model needs regular updates to maintain accuracy. Changes in vehicle conditions can affect predictions. Without retraining, performance may decrease over time. Continuous monitoring of model behavior is necessary. Periodic updates ensure consistent and reliable results.

3.2 Proposed System

This project introduces a **BrakeGuard-XAI – An Advanced Secure Explainable AI Paradigm for Early-Stage Brake Anomaly Detection and Interpretable Predictive Maintenance**. The system aims to predict brake faults at an early stage using sensor data and machine learning techniques.

The proposed system uses air pressure sensor data collected from the braking system. Data preprocessing techniques are applied to handle missing values and class imbalance.

The system uses the following machine learning model:

1. **Random Forest (RF)**
2. **Logistic Regression (LR)**
3. **Support Vector Machine (SVM)**
4. **Neural Networks (NN)**
5. **Gradient Boosting (GB)**
   By utilizing these models, the study illustrates how feature selection improves defect prediction accuracy across different machine learning algorithms.

3.2.1 Advantages of Proposed System

1. **Early Fault Detection**

   Predicts faults before failure occurs. Early fault detection in brake systems of heavy transport vehicles is crucial for ensuring safety and reliability. Identifying issues like worn brake pads, air pressure leaks, or overheating at an early stage helps prevent sudden brake failure, which could lead to serious accidents.

2. **Prioritizing Testing Efforts**

   Machine learning facilitates the identification of high-risk areas in software early in the development process, enabling testing teams to focus their efforts on parts of the software most likely to contain defects. This results in better resource allocation and more effective testing.

3. **Efficient Resource Allocation and Informed Decision-Making** Defect prediction helps software development teams allocate resources more efficiently by concentrating on high-risk modules or areas that require additional testing or code reviews.

4. **Enhanced Customer Satisfaction**

   By identifying and resolving defects before they reach end-users, defect prediction ensures that software is more stable and reliable, leading to greater customer satisfaction.

5. **Reduced Maintenance Costs**

   Early defect detection reduces the need for costly post-release maintenance. Addressing defects during the development phase can save valuable resources that would otherwise be spent on debugging and post-deployment fixes.

3.3 System Requirements Specification

- **Hardware Requirements:**

  - The hardware requirements specify the minimum and recommended system configuration needed to develop and run the proposed system efficiently.

  - **Processor**: Intel i5 or higher
  - **RAM**: Minimum 8 GB
  - **Storage**: At least 256 GB hard disk
  - **GPU (Optional):** Minimum 8 GB GPU for faster model training
  - The above hardware configuration ensures smooth execution of machine learning models, data processing, and visualization tasks.

  This configuration supports smooth execution of machine learning models, data processing, and visualization.

- **Software Requirements:**

  - The software requirements include the tools and technologies used to develop and implement the proposed system.
  - **Operating System**: Windows 10/11
  - **Programming Language**: Python 3.16
  - **Development Environment**: Jupyter Notebook / Visual Studio Code
  - **Libraries Used**:
    - **NumPy –** for numerical computations
    - **Pandas –** for data preprocessing and handling
    - **Scikit-learn** – for machine learning model (Random Forest)
  - **Explainable AI Tools**:
    - **SHAP –** for feature importance (global explanation)
    - **LIME** – for explaining individual predictions (local explanation)

  These software tools help in data analysis, model building, visualization, and explainability of predictions.

CHAPTER-4

SYSTEM DESIGN

CHAPTER-4

SYSTEM DESIGN

SYSTEM ARCHITECTURE:

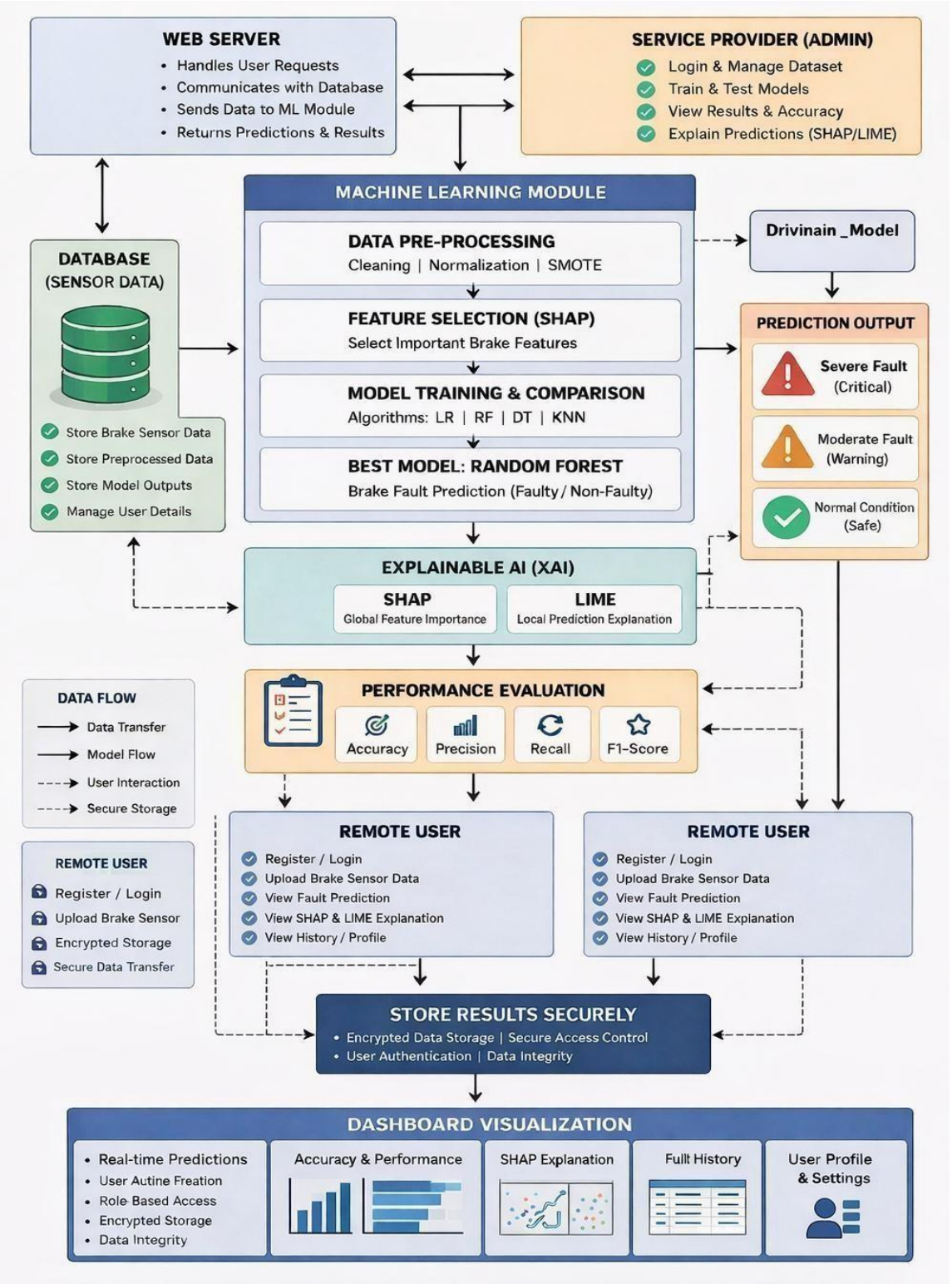

Fig. 4.1 System Architecture Diagram

DATA FLOW DIAGRAM:

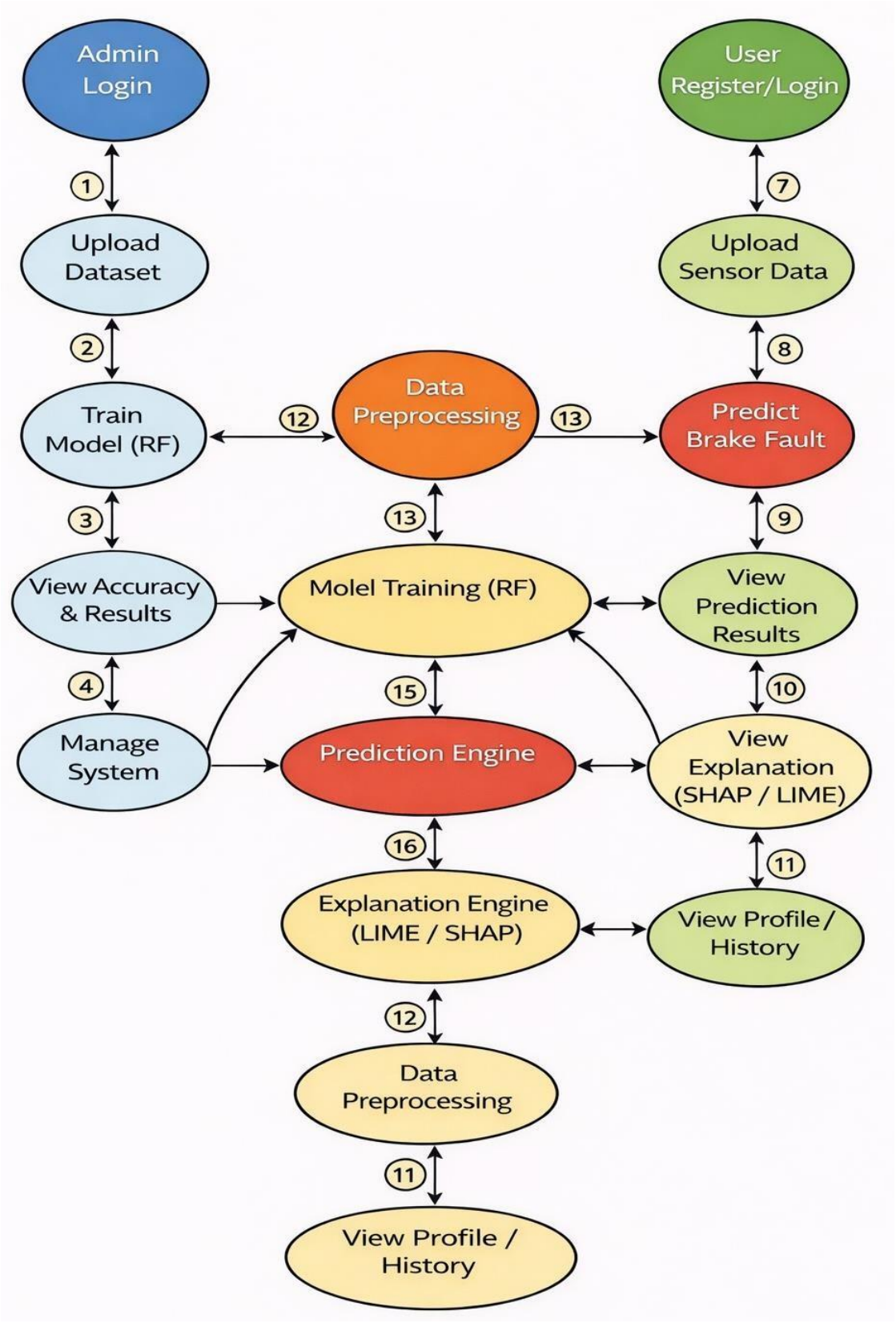

Fig. 4.2 Data Flow Diagram

FLOW CHART DIAGRAM :

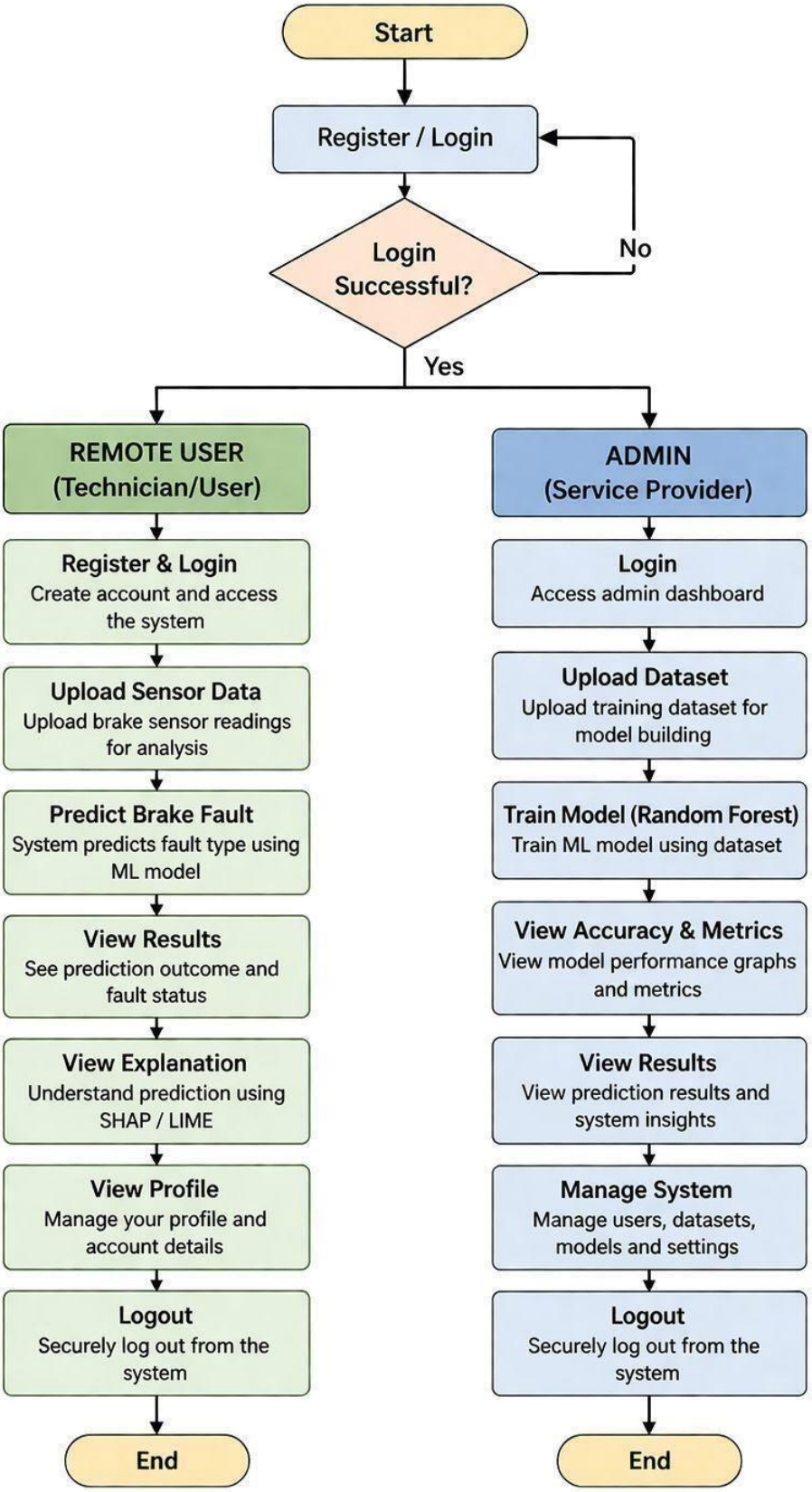

Fig. 4.3 Flow Chart Diagram

ACTIVITY DIAGRAM :

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step by-step workflows of components in a system. An activity diagram shows the overall flow of control.

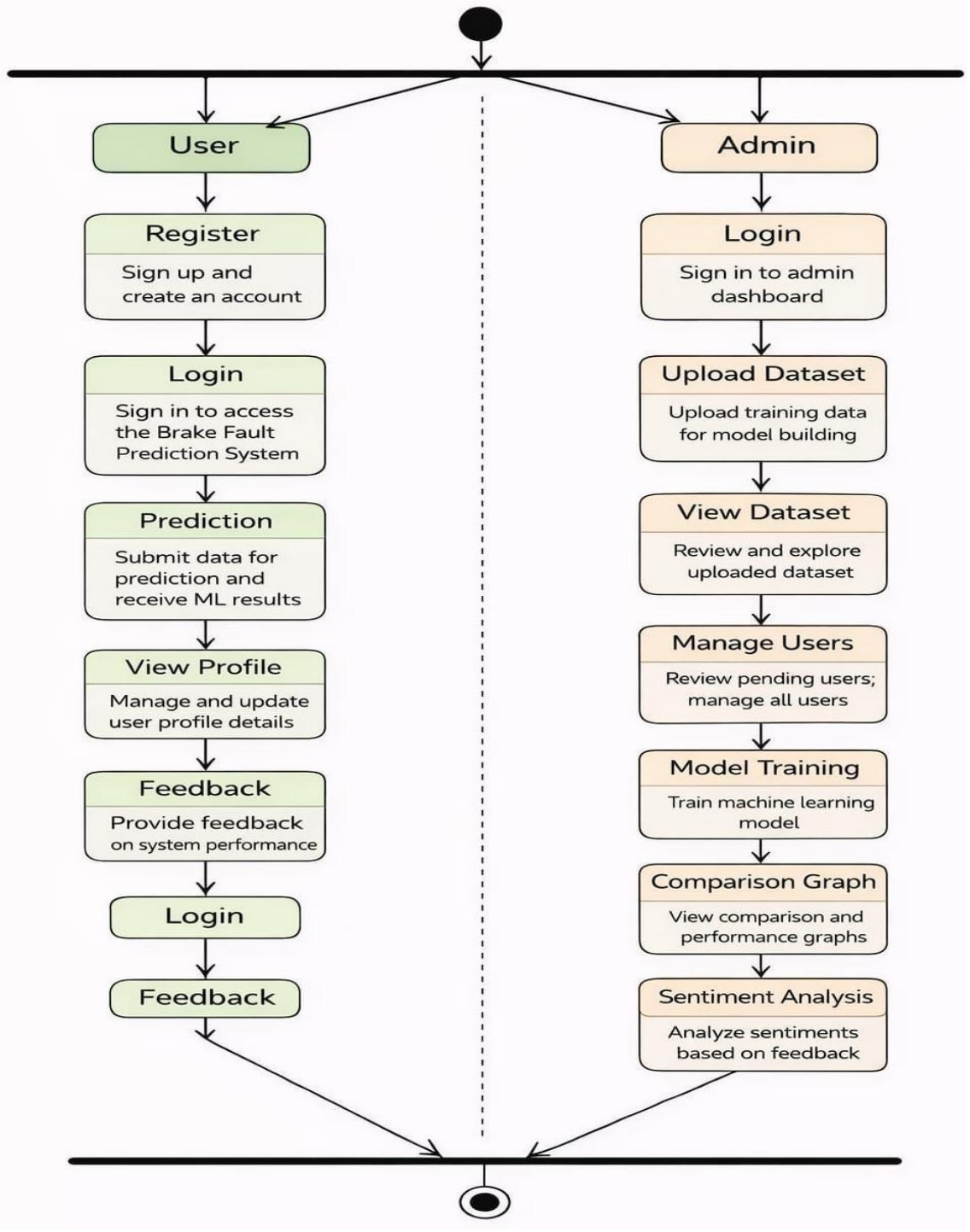

Fig. 4.4 Activity Diagram

CLASS DIAGRAM:

The **Class Diagram** is a core UML diagram that describes the static structure of a system. It shows the system's classes, their attributes (data fields), methods (functions or procedures), and the relationships between the classes. Class diagrams play a vital role in object-oriented design by providing a structural framework for organizing the system's components. They act as a blueprint for defining the system's architecture and help in efficiently structuring and organizing the system's elements.

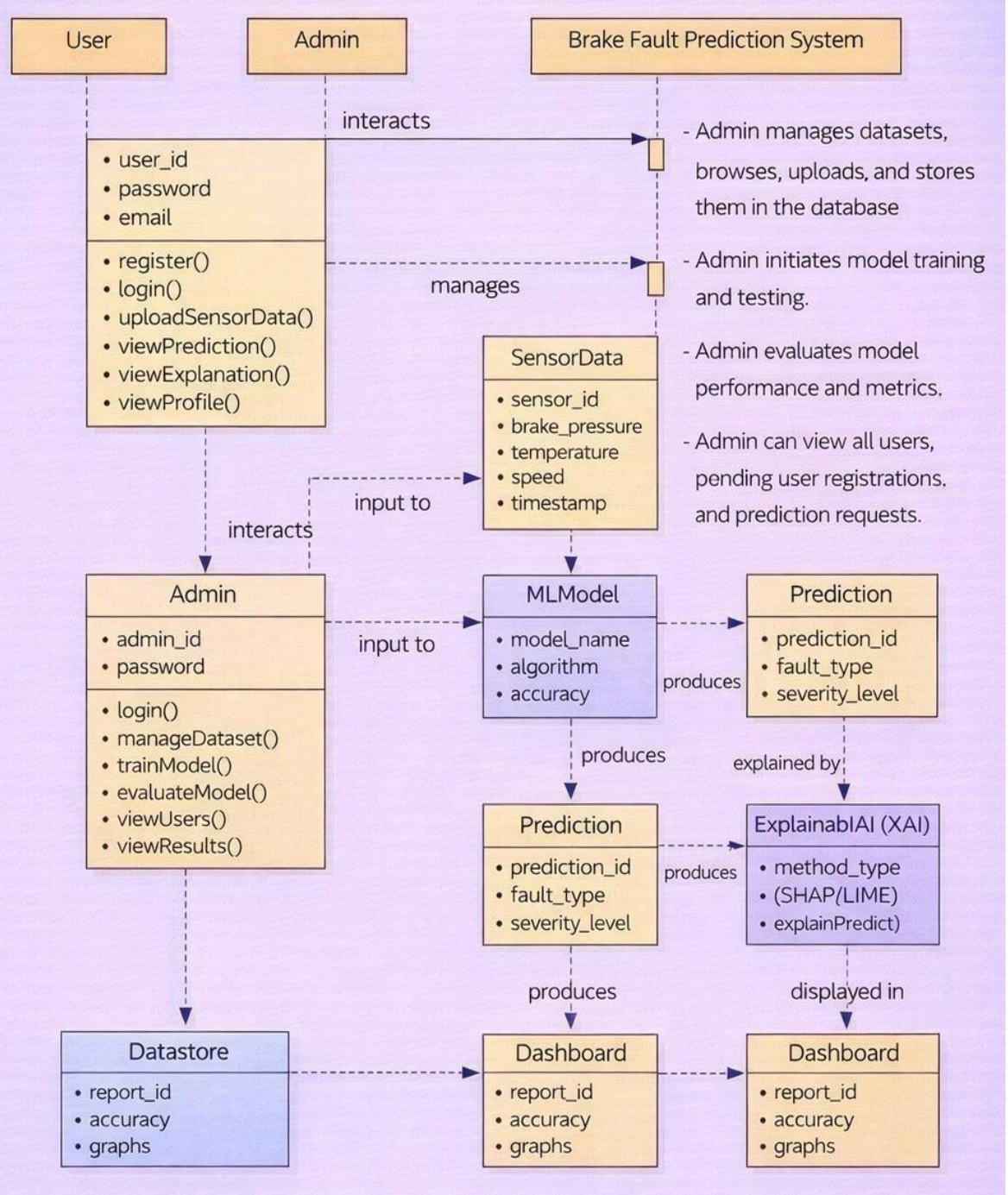

Fig. 4.5 Class Diagram

USE CASE DIAGRAM:

A **Use Case Diagram** is a type of behavioral diagram in the Unified Modeling Language (UML), derived from use-case analysis. It is designed to provide a visual representation of the functional requirements of a system, focusing on the interactions between **actors** (which can be users or other systems) and the **use cases** (functions or actions) of the system.

The primary goal of a use case diagram is to illustrate which system functions are available to each actor. It highlights the roles and responsibilities of the actors and demonstrates the relationships between them and the use cases they interact with. This approach helps clarify the system's intended behavior and outlines the goals that the actors aim to accomplish within the system.

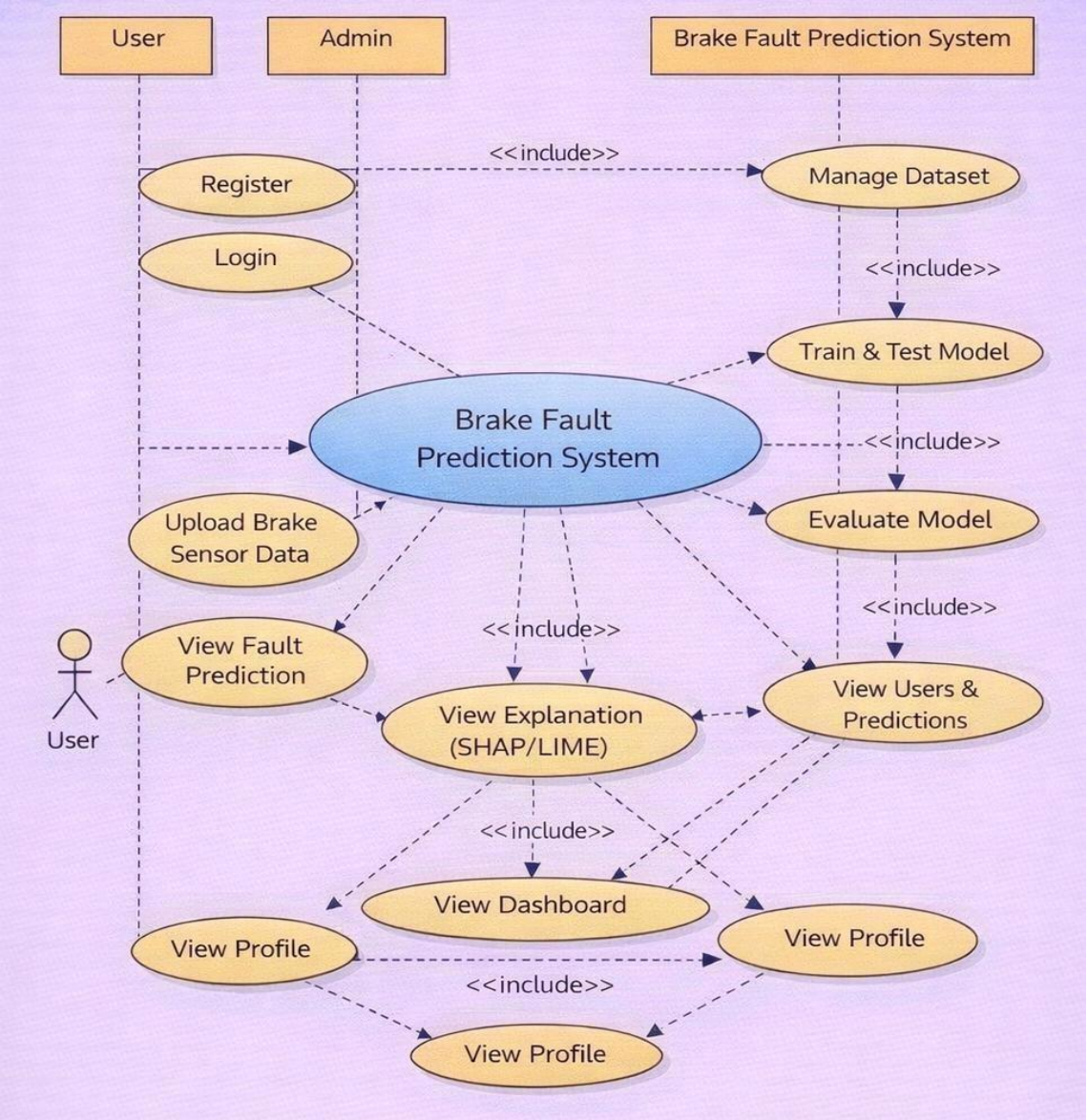

Fig. 4.6 Use Case Diagram

SEQUENCE DIAGRAM:

A **Sequence Diagram** in the Unified Modeling Language (UML) is an interaction diagram that illustrates the sequence of messages exchanged between various objects or processes in a system. It shows the order in which these interactions occur, capturing the dynamic flow of control and data throughout the system. Originating from the **Message Sequence Chart**, it is sometimes referred to as event diagrams, event scenarios, or timing diagrams.

The main purpose of a sequence diagram is to model the dynamic behavior of a system, providing a clear view of how objects collaborate over time to accomplish a specific task or goal. By visualizing the interactions and message flow, the diagram helps developers understand the system's execution order and the relationships between components involved in the process.

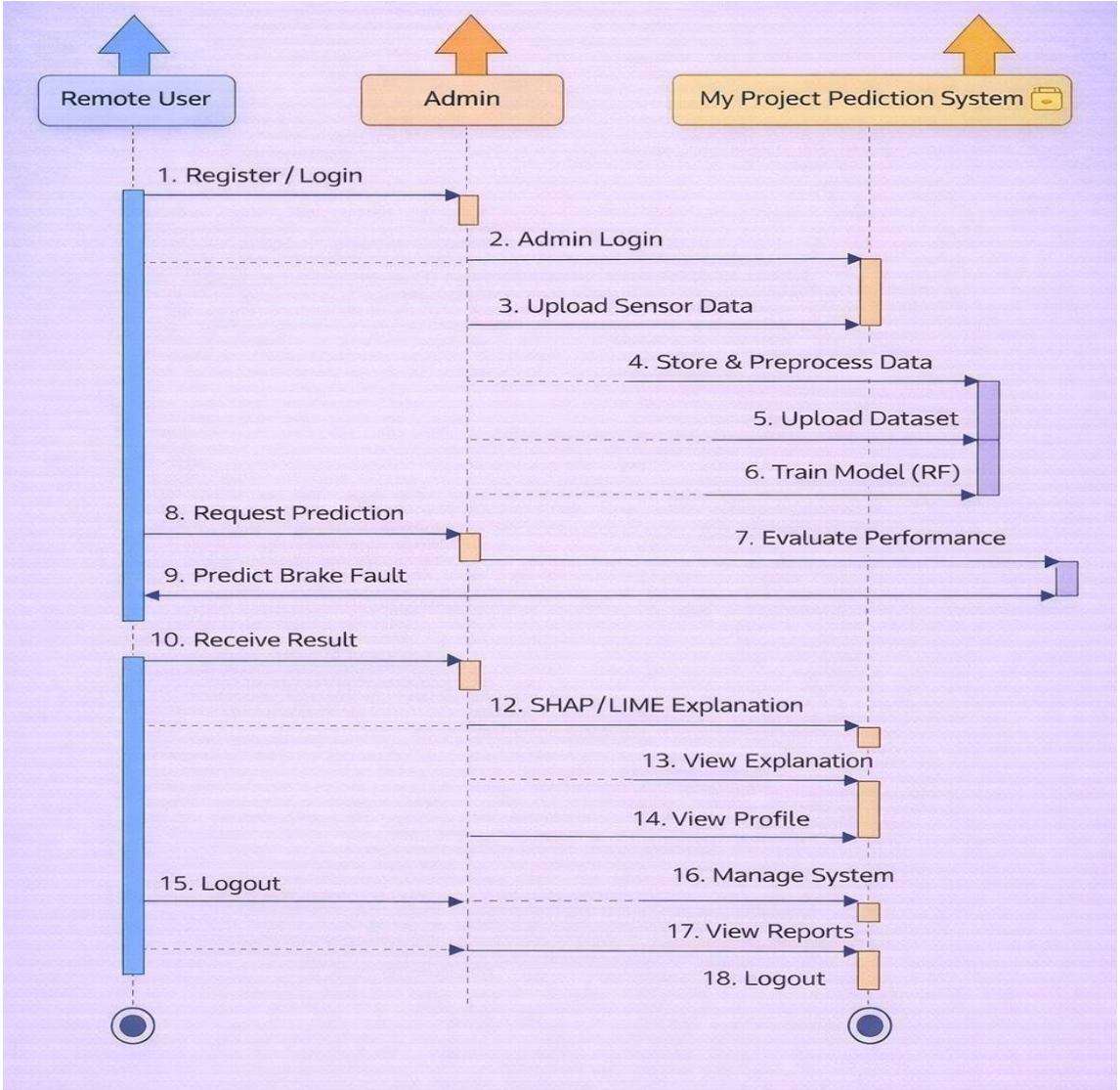

Fig. 4.7 Sequence Diagram

CHAPTER-5

IMPLEMENTATION

CHAPTER-5

IMPLEMENTATION

Modules

1. Service Provider Module: The Service Provider Module is responsible for managing datasets, training machine learning models, and analyzing system performance. It allows administrators to upload and browse brake system datasets containing sensor data such as air pressure values. Data pre-processing is performed by handling missing values, encoding data, and scaling features. Machine learning models like Random Forest are trained and tested using the dataset. Hyper-parameter tuning is applied to improve model performance. The module also provides evaluation metrics such as accuracy, precision, recall, and F1-score. Additionally, Explainable AI techniques like SHAP and LIME are used to explain model predictions, and results can be downloaded for further analysis.

2. **Remote User Module:** The Remote User Module allows users to interact with the system for brake fault prediction. Users can register and log in securely to access the system. They can upload sensor data as input, and the trained model predicts whether the brake system is faulty or not. The system displays prediction results along with explanations using SHAP and LIME, helping users understand the decision-making process. Users can also view their profile details and prediction history. This module ensures easy access and user-friendly interaction with the system.

3. **Data Pre-processing Module:** This module prepares the dataset for machine learning. It handles missing values using techniques such as KNN imputation and manages class imbalance using SMOTE. The data is normalized and scaled to improve model performance. Input features and output labels are separated, and the dataset is divided into training and testing sets. Proper preprocessing ensures accurate and efficient model training.

4. Model Implementation Module: This module implements machine learning algorithms for brake fault prediction. Models such as Random Forest, Decision Tree, Logistic Regression, KNN, and Naive Bayes are used. Among these, Random Forest provides the best performance. The models are trained using processed data and tested on unseen data to evaluate prediction capability.

5. **Model Evaluation Module:** This module evaluates the performance of trained models using metrics such as accuracy, precision, recall, and F1-score. Confusion matrices and graphs are used for visualization. The module helps in comparing different models and selecting the best-performing model for deployment.

6. **User Interface and Reporting Module:** This module provides a user-friendly interface for both service providers and users. It allows interaction with the system, displays prediction results, and generates reports. Visualizations such as graphs and charts are used to present model performance and results clearly.

Software Environment

Python Programming Language :

Python is selected as the primary programming language for the Secure Explainable AI-based Brake Fault Prediction System due to its strong capabilities in data processing, machine learning, and explainable AI. The project involves handling large volumes of sensor data such as air pressure, temperature, and mechanical parameters collected from heavy transport vehicles.

Role of Python in the Project :

In this project, Python is used to perform data preprocessing using libraries like NumPy and Pandas, which help in cleaning and organizing sensor data. It also supports the implementation of machine learning algorithms such as Decision Tree, Random Forest, and Support Vector Machine using Scikit-learn. These models analyze the processed data and predict whether the brake system is normal or faulty. Python further enables the use of Explainable AI techniques like SHAP and LIME, which provide insights into the predictions.

Key Features of Python:

1. **Interpreted Language**: Allows easy execution and debugging during model development.
2. **Object-Oriented Programming**: Helps in organizing the system into structured modules.
3. **Rich Library Support**: Provides powerful libraries for data analysis, machine learning, and visualization.

4. **Rich Library Support**: Provides powerful libraries for data analysis, machine learning, and visualization.
5. **Cross-Platform Compatibility**: Runs on different operating systems without modification.
6. **Scalability**: Efficiently handles large datasets and supports system expansion.

Django Framework

Django is a high-level Python web framework used for developing secure, scalable, and efficient web applications. In the Secure Explainable AI-based Brake Fault Prediction System, Django is used to build the web interface that allows users to interact with the system, upload data, and view prediction results.

Role of Django in the Project:

In this project, Django is used to develop the frontend and backend of the web application. It provides a platform where users (technicians or administrators) can upload sensor data, trigger the prediction process, and view the results. Django handles the communication between the user interface and the machine learning models developed in Python. It also manages data storage, user requests, and displays prediction outputs along with explainable results in an organized manner.

Key Features of Django:

1. **Rapid Development**: Django supports rapid development by providing built-in features such as URL routing, form handling, and admin panels. This feature helps in quickly developing the web interface where users can upload sensor data and view brake fault prediction results. It reduces the time required to build the system and allows faster implementation of the project.
2. **Database Integration**: Django provides an Object-Relational Mapper (ORM) that allows interaction with databases using Python code instead of writing complex SQL queries.
3. **Security**: Django includes built-in security features to protect the application from common web attacks such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).
4. **User Authentication**: Django provides a built-in authentication system that manages user login, logout, and access control. It is used to restrict system acess

to authorized users such as administrators or technicians. This ensures that only valid users can upload data, view predictions, and manage the system.

5. **Modular Design (Reusability)**: Django follows a modular architecture, allowing developers to create reusable components. The modules like data upload forms, dashboards, and result display pages can be reused and easily managed. This improves code organization, reduces development time, and makes the system easier to maintain and update.

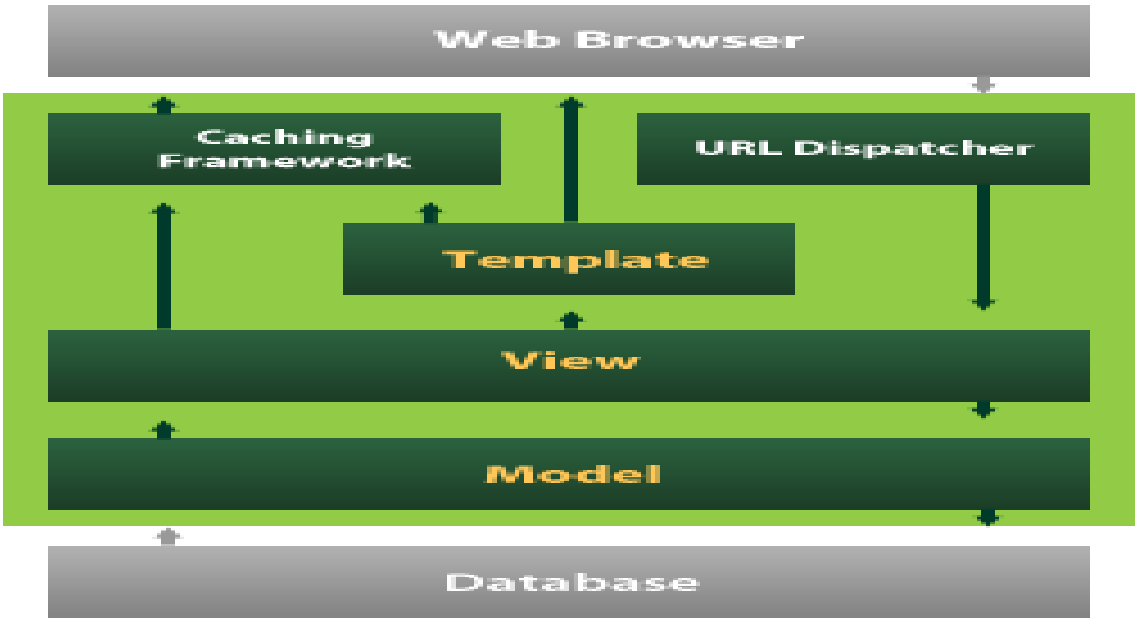

Fig. 5.1 Django Architecture Diagram

"Django provides an optional admin interface that allows for the CRUD operations (Create, Read, Update, Delete) on data. This interface is automatically generated from models and can be customized using admin models for specific configurations."

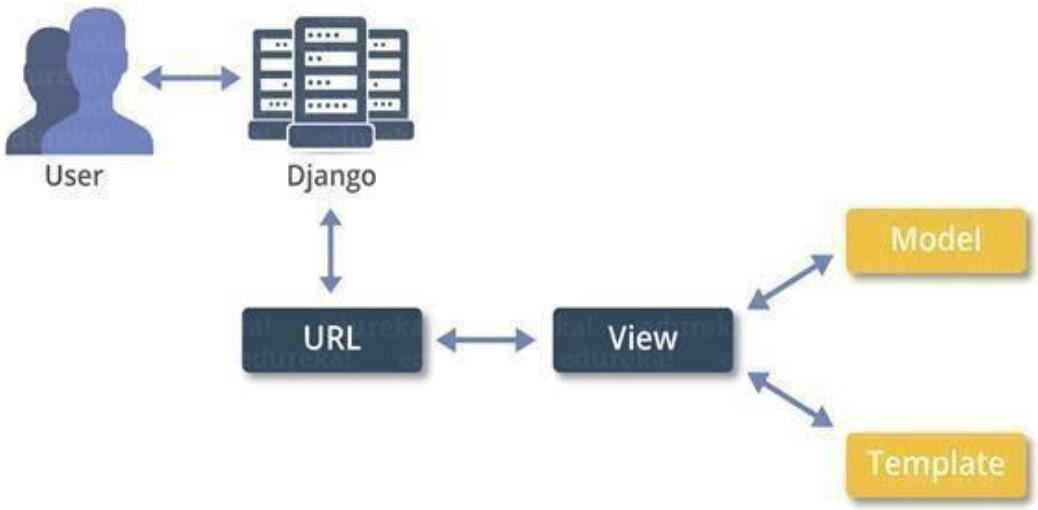

Fig. 5.2 Django Working Flow Diagram

Service Provider Sample Python Code:

```
# Create your views here.
from django.shortcuts import render,redirect
from mainapp.models import*
from userapp.models import*
from adminapp.models import *
from  django.contrib  import  messages
from django.core.paginator import Paginator
from django.http import HttpResponse
import os
import  shutil
from django.shortcuts import render
import pandas as pd
import  numpy as  np
from  sklearn.model_selection  import  train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from django.contrib import messages
from keras.models  import  load_model
def adminlogout(req):
    messages.info(req,'You are logged out...!')
    return redirect('admin')
def admindashboard(req):
    all_users_count =   User.objects.all().count()
    pending_users_count = User.objects.filter(User_Status = 'Pending').count()
    rejected_users_count = User.objects.filter(User_Status = 'removed').count()
    accepted_users_count =User.objects.filter(User_Status = 'accepted').count()
    Feedbacks_users_count= Feedback.objects.all().count()
    user_uploaded_images  =Dataset.objects.all().count()
    return render(req,'admin/admin-dashboard.html',{'a' : all_users_count, 'b' :
pending_users_count, 'c' : rejected_users_count, 'd'  : accepted_users_count,
'e':Feedbacks_users_count,  'f':user_uploaded_images})
def pendingusers(req):
    pending = User.objects.filter(User_Status = 'Pending')
    paginator = Paginator(pending, 5)
```

```
    page_number = req.GET.get('page')
    post = paginator.get_page(page_number)
    return render(req,'admin/admin-pending-users.html', { 'user' : post})
def delete_user(req, id):
    User.objects.get(User_id = id).delete()
    messages.warning(req, 'User was Deleted..!')
    return redirect('manageusers')
def accept_user(req, id):
    status_update = User.objects.get(User_id = id)
    status_update.User_Status = 'accepted'
    status_update.save()
    messages.success(req, 'User was accepted..!')
    return redirect('pendingusers')
def manageusers(req):
    manage_users = User.objects.all()
    paginator = Paginator(manage_users, 5)
    page_number = req.GET.get('page')
    post = paginator.get_page(page_number)
    return render(req, 'admin/admin-manage-users.html', {"allu" : manage_users, 'user'
: post})
def reject_user(req, id):
    status_update2 = User.objects.get(User_id = id)
    status_update2.User_Status = 'removed'
    status_update2.save()
    messages.warning(req, 'User was Rejected..!')
    return redirect('pendingusers')
def admin_datasetupload(req):
     return render(req,'admin/admin-upload-dataset.html')
def admin_dataset_btn(req):
    messages.success(req, 'Dataset uploaded successfully..!')
    return redirect('admin_datasetupload')
def adminfeedback(req):
    feed =Feedback.objects.all()
    return render(req,'admin/user-feedback.html', {'back':feed})
def adminsentiment(req):
```

```
    fee = Feedback.objects.all()
    return render(req,'admin/user-sentiment.html' , {'cat':fee})
def usergraph(req):
    positive = Feedback.objects.filter(Sentiment = 'positive').count()
    very_positive = Feedback.objects.filter(Sentiment = 'very positive').count()
    negative = Feedback.objects.filter(Sentiment = 'negative').count()
    very_negative = Feedback.objects.filter(Sentiment = 'very negative').count()
    neutral = Feedback.objects.filter(Sentiment = 'neutral').count()
    context ={
        'vp': very_positive, 'p':positive, 'n':negative, 'vn':very_negative, 'ne':neutral }
    return render(req,'admin/user-sentiment-graph.html',context)
def RF_alg(req):
  return render(req,'admin/Random_forest.html')
def DT_alg(req):
  return render(req,'admin/Decision_tree.html')
def LR_alg(req):
  return render(req,'admin/Logistic.html')
def GBoost_alg(req):
  return render(req,'admin/GBoost.html')
def RF_btn(req):
    import pickle
    import pandas as pd
    from sklearn.metrics import accuracy_score, precision_score, recall_score,
fbeta_score
    from sklearn.preprocessing import StandardScaler
    from sklearn.model_selection import train_test_split
    with open('rf_model_all_features.pkl', 'rb') as model_file:
        rf_model_all_features = pickle.load(model_file)
with open('rf_model_top_features.pkl', 'rb')
   as model_file_top: rf_model_top_features = pickle.load(model_file_top)
   train_df = pd.read_csv('C:/Users/system -
   2/Desktop/Break_fault_in_Heavy_Transports/Dataset/aps_failure_training_set.csv',
   na_values=["na"])
    test_df = pd.read_csv('C:/Users/system2/Desktop/Break_fault_in_Heavy_
Transports/Dataset/aps_failure_test_set.csv',na_values=["na"])
df = pd.concat([train_df, test_df], ignore_index=True)
```

```
df['class'] = df['class'].apply(lambda x: 1 if x == 'pos' else 0)
df = df.apply(pd.to_numeric, errors='coerce')
df = df.fillna(df.mean())

X = df.drop(columns=['class'])
y = df['class']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
y_pred_all_features = rf_model_all_features.predict(X_test_scaled)
accuracy_all_features = accuracy_score(y_test, y_pred_all_features)*100
precision_all_features = precision_score(y_test, y_pred_all_features)*100
recall_all_features = recall_score(y_test, y_pred_all_features)*100
f2_all_features = fbeta_score(y_test, y_pred_all_features, beta=2)*100
feature_importances = rf_model_top_features.feature_importances_
top_features_indices = feature_importances.argsort()[-170:][::-1]  # Get indices
of top 170 features
top_features = X.columns[top_features_indices].tolist()
print("Top features:", top_features)
X_test_top_features = X_test[top_features]

print("Selected columns in X_test:", X_test_top_features.columns)
X_train_top_features = X_train[top_features]
X_train_top_features_scaled = scaler.fit_transform(X_train_top_features)

y_pred_top_features = rf_model_top_features.predict(X_test_top_features_scaled)
accuracy_top_features = accuracy_score(y_test, y_pred_top_features)*100
precision_top_features = precision_score(y_test, y_pred_top_features)*100
recall_top_features = recall_score(y_test, y_pred_top_features)*100
f2_top_features = fbeta_score(y_test, y_pred_top_features, beta=2)*100
name = "Random Forest Algorithm"
RF.objects.create(accuracy=accuracy_top_features,
Precession=precision_top_features, f1_score=f2_top_features,
recall=recall_top_features, Name=name)
data = RF.objects.last()
```

```
    messages.success(req, 'Algorithm executed successfully')
    return render(req, 'admin/Random_forest.html', {
        'accuracy':    accuracy_top_features,
        'precision':   precision_top_features,
        'recall': recall_top_features,
        'f1': f2_top_features,
        'feature1': top_features[0],
        'feature2': top_features[1],
        'Name':name })
def DT_btn(req):
    import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.metrics import accuracy_score, precision_score, recall_score,
fbeta_score
    from sklearn.preprocessing import StandardScaler  # Import StandardScaler for
feature scaling
    import shap
    import pickle
train_df = pd.read_csv('C:/Users/system -
2/Desktop/Break_fault_in_Heavy_Transports/Dataset/aps_failure_training_set.csv',
na_values=["na"])
    test_df = pd.read_csv('C:/Users/system -
2/Desktop/Break_fault_in_Heavy_Transports/Dataset/aps_failure_test_set.csv',
na_values=["na"])
    df = pd.concat([train_df, test_df], ignore_index=True)
    df['class'] = df['class'].apply(lambda x: 1 if x =='pos’else 0)  df =
    df.apply(pd.to_numeric, errors='coerce')
    df = df.fillna(df.mean())
    X = df.drop(columns=['class'])  #  Assuming 'class' is the target  column
    y = df['class']  # Encoded target variable
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
    scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_scaled, y_train)
y_pred = dt_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)*100
precision = precision_score(y_test, y_pred)*100
recall = recall_score(y_test, y_pred)*100
f2 = fbeta_score(y_test, y_pred, beta=2)*100
X_test_scaled = scaler.transform(X_test)
explainer = shap.TreeExplainer(dt_model)
shap_values = explainer.shap_values(X_test_scaled)
shap_importances = pd.DataFrame(list(zip(X.columns,
abs(shap_values[1]).mean(axis=0))), columns=['feature', 'importance'])
top_features_count = 30 # Select the top 30 features, you can change this number
top_features = shap_importances.sort_values(by='importance',
ascending=False).head(top_features_count)['feature'].tolist()
print(f"Top {top_features_count} Features based on SHAP values:", top_features)
X_train_top_features = X_train[top_features]
X_test_top_features = X_test[top_features]
X_train_top_features_scaled = scaler.fit_transform(X_train_top_features)
X_test_top_features_scaled = scaler.transform(X_test_top_features)
dt_model_top_features = DecisionTreeClassifier(random_state=42) # Train
Decision Tree with top features
dt_model_top_features.fit(X_train_top_features_scaled,      y_train)
y_pred_top_features = dt_model_top_features.predict(X_test_top_features_scaled)
accuracy_top_features = accuracy_score(y_test, y_pred_top_features)*100
precision_top_features = precision_score(y_test, y_pred_top_features)*100
recall_top_features = recall_score(y_test, y_pred_top_features)*100
f2_top_features = fbeta_score(y_test, y_pred_top_features, beta=2)*100
name = "Decision Tree Algorithm"
DT.objects.create(accuracy=accuracy_top_features,
Precession=precision_top_features, f1_score=f2_top_features,
recall=recall_top_features, Name=name)
data = DT.objects.last()
```

```
messages.success(req, 'Algorithm executed successfully')
return render(req, 'admin/Decision_tree.html', {
    'accuracy': accuracy_top_features,
    'precision': precision_top_features,
    'recall': recall_top_features,
    'f1': f2_top_features,
    'feature1': top_features[0],
    'feature2': top_features[1],
    'Name':name })
    def LR_btn(req): import
    pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
 from sklearn.metrics import accuracy_score, precision_score, recall_score
 fbeta_score
 from sklearn.preprocessing import StandardScaler
 import shap
 import pickle
 train_df = pd.read_csv('C:/Users/system - 2/Desktop/Break_fault_in_Heavy
 _Transports/Dataset/aps_failure_training_set.csv', na_values=["na"])
test_df = pd.read_csv('C:/Users/system - 2/Desktop/Break_fault_in_Heavy
_Transports/Dataset/aps_failure_test_set.csv', na_values=["na"])
df = pd.concat([train_df, test_df], ignore_index=True)
X_test_scaled = scaler.transform(X_test)
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)
y_pred = log_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)*100
precision = precision_score(y_test, y_pred)*100
recall = recall_score(y_test, y_pred)*100
f2 = fbeta_score(y_test, y_pred, beta=2)*100
explainer = shap.LinearExplainer(log_model, X_train_scaled) # Use
LinearExplainer for Logistic Regression
shap_values = explainer.shap_values(X_test_scaled)
```

```
    shap_importances = pd.DataFrame(list(zip(X.columns,
abs(shap_values).mean(axis=0))), columns=['feature', 'importance'])
top_features_count = 30 # Select the top 30 features, you can change this
  number top_features = shap_importances.sort_values(by='importance',
ascending=False).head(top_features_count)['feature'].tolist()
    print(f"Top {top_features_count} Features based on SHAP values:", top_features)
    X_train_top_features = X_train[top_features]
    X_test_top_features = X_test[top_features]
   X_train_top_features_scaled = scaler.fit_transform(X_train_top_features)
   X_test_top_features_scaled = scaler.transform(X_test_top_features)
    log_model_top_features = LogisticRegression(max_iter=1000, random_state=42)
    log_model_top_features.fit(X_train_top_features_scaled, y_train)
    y_pred_top_features =
    log_model_top_features.predict(X_test_top_features_scaled)
accuracy_top_features = accuracy_score(y_test, y_pred_top_features)*100
precision_top_features = precision_score(y_test, y_pred_top_features)*100
recall_top_features = recall_score(y_test, y_pred_top_features)*100
f2_top_features = fbeta_score(y_test, y_pred_top_features, beta=2)*100
    name = "Logistic Algorithm"
    LR.objects.create(accuracy=accuracy_top_features,
Precession=precision_top_features, f1_score=f2_top_features,
recall=recall_top_features, Name=name)

    data = LR.objects.last()
    messages.success(req, 'Algorithm executed successfully') return render(req,
    'admin/Logistic.html', {
        'top_feature_accuracy': accuracy_top_features,
        'top_feature_precision': precision_top_features,
        'top_feature_recall': recall_top_features,
        'top_feature_f1': f2_top_features,
        'feature1':   top_features[0],
        'feature2':   top_features[1],
        'feature3':   top_features[2],
        'feature4':   top_features[3],
        'feature5':   top_features[4],
        'Name': name })
```

```
def   GBoost_btn(req):
    import pandas as pd
    import pickle
    from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
    train_df = pd.read_csv('C:/Users/system - 2/Desktop/Break_fault_in_Heavy
_Transports/Dataset/aps_failure_training_set.csv', na_values=["na"])
    test_df = pd.read_csv('C:/Users/system - 2/Desktop/Break_fault_in_Heavy
_Transports/Dataset/aps_failure_test_set.csv', na_values=["na"])
   df = pd.concat([train_df, test_df], ignore_index=True)
   df['class'] = df['class'].apply(lambda x: 1 if x == 'pos' else 0)
   df = df.apply(pd.to_numeric, errors='coerce')
    df = df.fillna(df.mean())
    X = df.drop(columns=['class'])
    y = df['class']
   with open('gradient_boosting_model.pkl', 'rb') as model_file:
   gb_model = pickle.load(model_file)
   with open('gradient_boosting_top_features_model.pkl', 'rb') as
top_features_model_file:gb_model_top_features = pickle.load(top_features_model_file)
    with open('scaler.pkl', 'rb') as scaler_file: scaler =
    pickle.load(scaler_file)
    X_full_ordered = X[scaler.feature_names_in_]
    X_full_scaled = scaler.transform(X_full_ordered)
    y_pred_gb_model = gb_model.predict(X_full_scaled)
    accuracy_gb_model = accuracy_score(y, y_pred_gb_model)
    precision_gb_model = precision_score(y, y_pred_gb_model)
    recall_gb_model = recall_score(y, y_pred_gb_model)
    f1_gb_model = f1_score(y, y_pred_gb_model)
    print("Gradient Boosting Model (All Features) Performance:")
    print(f"Accuracy: {accuracy_gb_model * 100:.2f}%")
    print(f"Precision: {precision_gb_model * 100:.2f}")
    print(f"Recall: {recall_gb_model * 100:.2f}")
    print(f"F1 Score: {f1_gb_model * 100:.2f}")
   top_features=['aa_000', 'bt_000', 'ay_008', 'ag_002', 'ag_006', 'am_0', 'ci_000',
   'ck_000', 'cc_000', 'az_002', 'bj_000', 'ai_000', 'ay_006', 'cn_004', 'ee_005', 'dx_000',
   'cn_001', 'dt_000', 'ay_009', 'ee_007', 'bs_000', 'ay_005', 'do_000', 'al_000', 'aq_000',
```

```
'dg_000', 'bc_000', 'ba_007', 'ag_001', 'cn_002', 'ay_000', 'ao_000', 'an_000', 'ag_004',
'cg_000', 'cp_000', 'bx_000', 'az_000', 'de_000', 'ah_000', 'ag_005', 'cj_000', 'ba_001',
'dv_000', 'ba_000', 'ad_000', 'ay_001', 'cs_003', 'ee_003', 'cn_003', 'ba_006', 'bh_000',
'cm_000', 'ay_002', 'aj_000', 'bk_000', 'cl_000', 'ay_007', 'ee_008', 'cs_005', 'bq_000',
'bg_000', 'dy_000', 'bi_000', 'dd_000', 'dr_000', 'di_000', 'az_001', 'ag_007', 'bn_000',
'au_000', 'bf_000', 'cn_007', 'az_004', 'az_003', 'bl_000', 'ba_009', 'cy_000', 'ag_009',
'ee_009', 'cs_004', 'ee_004', 'db_000', 'bz_000', 'br_000', 'du_000', 'as_000', 'bb_000',
'co_000', 'cz_000', 'ec_00', 'ee_001', 'dq_000', 'bp_000', 'bo_000', 'dl_000', 'ay_003',
'ca_000', 'ac_000', 'ba_003', 'dj_000', 'ay_004', 'ee_000', 'az_006', 'ba_008', 'ce_000',
'ag_003', 'ba_002', 'ba_004', 'ds_000', 'at_000', 'dk_000', 'cr_000', 'cn_000', 'be_000',
'bm_000', 'az_007', 'cs_009', 'cs_008', 'cs_006', 'ct_000', 'cn_009', 'ed_000', 'ef_000',
'cx_000', 'dz_000', 'ag_008', 'az_005', 'cb_000', 'ee_002', 'av_000', 'ap_000', 'by_000',
'cs_007', 'ab_000', 'ae_000', 'af_000', 'ag_000', 'ar_000', 'ax_000', 'ak_000', 'ch_000',
'cn_006', 'cn_005', 'cd_000', 'bd_000', 'ba_005', 'az_008', 'az_009', 'cf_000', 'bv_000',
'bu_000', 'cn_008', 'cq_000', 'dh_000', 'df_000', 'dc_000', 'da_000', 'cv_000', 'cu_000',
'cs_001', 'cs_002', 'cs_000', 'dn_000', 'dp_000', 'dm_000', 'eb_000', 'ea_000', 'ee_006',
'eg_000']
X_top_features_ordered = X[top_features]
X_top_features_scaled = scaler.transform(X_top_features_ordered)
y_pred_top_features_model =
gb_model_top_features.predict(X_top_features_scaled)
accuracy_top_features_model = accuracy_score(y, y_pred_top_features_model)*100
precision_top_features_model = precision_score(y, y_pred_top_features_model)*100
recall_top_features_model = recall_score(y, y_pred_top_features_model)*100
f1_top_features_model = f1_score(y, y_pred_top_features_model)*100
  name = "GBoost Algorithm"
  GBoost.objects.create(accuracy=accuracy_top_features_model,
Precession=precision_top_features_model, f1_score=f1_top_features_model,
recall=recall_top_features_model, Name=name)
  data = GBoost.objects.last()
  messages.success(req, 'Algorithm executed successfully') return render(req,
'admin/GBoost.html', {
      'Name':name,
      'accuracy':    accuracy_top_features_model,
      'precision':   precision_top_features_model,
      'recall': recall_top_features_model,
      'f1': f1_top_features_model,
      'feature1': top_features[0],
     'feature2': top_features[1],
      'feature3': top_features[2],
```

```
        'feature4':  top_features[3],
         'feature5':  top_features[4],
         })
def admin_graph(request):
    details1 = RF.objects.last()
    rf = details1.accuracy
    details2 = DT.objects.last()
    dt = details2.accuracy
    details3 = LR.objects.last()
    lr = details3.accuracy
    details4 = GBoost.objects.last()
    gboost = details4.accuracy
    print('RF','DT','LR','GBoost')
    return render(request,'admin/admin-graph-analysis.html',{'RF':rf,'DT':dt,'LR':lr,
'GBoost':gboost })
```

Remote User Sample Code:

 # Create your views here.

```
from django.shortcuts import render,redirect
from mainapp.models import *
from userapp.models import Feedback,Dataset
from adminapp.models import *
from django.contrib import messages
import time
from django.core.paginator import Paginator
from vaderSentiment.vaderSentiment
import Sentiment Intensity Analyzer
from django.core.files.storage import default_storage
from django.conf import settings
import numpy as np
import numpy as np
import os
def userdashboard(req):
    images_count =  User.objects.all().count()
    print(images_count)
```

```
        user_id = req.session["User_id"]
        user = User.objects.get(User_id = user_id)
        return render(req,'user/user-dashboard.html', {'detect' :
images_count, 'la' : user})
    def profile(req):
        user_id = req.session["User_id"]
        user = User.objects.get(User_id = user_id)
        if req.method == 'POST':
            user_name = req.POST.get('userName')
            user_age = req.POST.get('userAge')
            user_phone = req.POST.get('userPhNum')
            user_email = req.POST.get('userEmail')
            user_address = req.POST.get("userAddress")
            user.Full_name = user_name
            user.Age = user_age
            user.Address = user_address
            user.Phone_Number = user_phone
            user.Email=user_email
            if len(req.FILES) != 0:
           image = req.FILES['profilepic']
                user.Image = image
                user.Full_name = user_name
                user.Age = user_age
                user.Address = user_address
                user.Phone_Number = user_phone
                user.Email=user_email
                user.Address=user_address
                user.save()

                messages.success(req, 'Updated SUccessfully...!')
            else:
                user.Full_name = user_name user.Age = user_age user.save()
                    messages.success(req, 'Updated SUccessfully...!')
        context = {"i":user}
        return render(req, 'user/user-profile.html', context)
```

```
def userlogout(req):
    user_id = req.session["User_id"]
    user = User.objects.get(User_id = user_id)
    t = time.localtime()
    user.Last_Login_Time = t
    current_time = time.strftime('%H:%M:%S', t)
    user.Last_Login_Time = current_time
    current_date = time.strftime('%Y-%m-%d')
    user.Last_Login_Date = current_date
    user.save()
    messages.info(req, 'You are logged out..')
    return redirect('login')
from tensorflow.keras.models import load_model
from  tensorflow.keras.optimizers  import  Adam
def userfeedbacks(req):
    id=req.session["User_id"]
    uusser=User.objects.get(User_id=id)
    if req.method == "POST":
        rating=req.POST.get("rating")
        review=req.POST.get("review")
        if not rating:
                messages.info(req,'give rating')
            return     redirect('userfeedbacks')
            sid=SentimentIntensityAnalyzer()
            score=sid.polarity_scores(review)
        sentiment=None
        if score['compound']>0 and score['compound']<=0.5:
            sentiment='positive'
         elif score['compound']>=0.5:
            sentiment='very  positive'
          elif score['compound']<-0.5:
             sentiment='very  negative'
        elif score['compound']<0 and score['compound']>=-0.5:
            sentiment='negative'
```

```
            else :
                sentiment='neutral'
            Feedback.objects.create(Rating=rating, Review=review,
            Sentiment=sentiment, Reviewer=uusser)
            messages.success(req,'Feedback recorded')
            return redirect('userfeedbacks')
        return render(req,'user/user-feedbacks.html')
    from django.shortcuts import render
    import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import GradientBoostingClassifier
    from sklearn.metrics import accuracy_score, precision_score,
recall_score, fbeta_score
    from sklearn.preprocessing import StandardScaler
    import shap
    import lime
    from lime import lime_tabular
    from django.shortcuts import render
    import joblib # Import joblib to load the model
    import pandas as pd
    from sklearn.metrics import accuracy_score, precision_score,
recall_score, fbeta_score
    import shap
    import lime
    from lime import lime_tabular
    from sklearn.preprocessing import StandardScaler
    from sklearn.model_selection import train_test_split
    import pickle
    def user_Gradiant_Boost(request):
        if request.method == 'POST':
        train_df =pd.read_csv('filtered_aps_failure_training_set.csv', na_values=["na"])
  test_df = pd.read_csv('filtered_aps_failure_test_set.csv',
            na_values=["na"])
            train_df.columns = train_df.columns.str.strip()
            test_df.columns = test_df.columns.str.strip()
```

```
df = pd.concat([train_df, test_df], ignore_index=True)
df['class'] = df['class'].apply(lambda x: 1 if  x == 'pos' else 0)
df = df.apply(pd.to_numeric, errors='coerce').fillna(df.mean())
X = df.drop(columns=['class'])
y = df['class']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
user_input = {
    'aa_000':  int(request.POST.get('aa_000',  0)),
    'ag_002':  int(request.POST.get('ag_002',  0)),
    'ai_000':  int(request.POST.get('ai_000',  0)),
    'am_000':  int(request.POST.get('am_000',  0)),
    'bj_000':  int(request.POST.get('bj_000',  0)),
    'cc_000':  int(request.POST.get('cc_000',  0)) }
input_data = pd.DataFrame(0, index=[0],columns=X.columns)
for feature, value in user_input.items():
    if feature in input_data.columns:
        input_data[feature] = value
    scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
input_data_scaled = scaler.transform(input_data)
with open('gradient_boostinggg_model.pkl', 'rb')
as  f: gb_model = pickle.load(f)
y_pred = gb_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred) * 100
precision = precision_score(y_test, y_pred) * 100
recall = recall_score(y_test, y_pred) * 100
f2 = fbeta_score(y_test, y_pred, beta=2) * 100
explainer = shap.Explainer(gb_model, X_train_scaled)
shap_values = explainer(input_data_scaled)
top_features = pd.DataFrame(list(zip(X.columns, abs(shap_values.values[0]))),
        columns=['feature', 'importance']).nlargest(5, 'importance')
solutions = {
'aa_000': "Check brake cylinder for air pressure issues.",
```

```
'cc_000': "Inspect control valves  for  leakage.",
    'am_000': "Verify actuator motor for functionality.",
    'ag_002': "Check brake pads for wear.",
    'bj_000': "Inspect wheel hub assembly for alignment.",
    'ai_000': "Check ABS system for functionality.",
    'bt_000': "Inspect air hoses for leaks or blockages.",
    'ci_000': "Verify pressure sensors and wiringintegrity.",
 'ck_000': "Check clutch mechanism and pressure system.",
'al_000': "Inspect braking system for wear and alignment.",
'aq_000': "Check actuator system for irregular behavior.",
'ap_000':  "Ensure  hydraulic  systems  are  leak-free.",
'bh_000': "Inspect battery connections and charge levels.",
'ag_001': "Check brake pads for wear and replaceifnecessary.",
'bg_000': "Ensure gear engagement system is functioning properly.",
'cn_001': "Check electronic control unit (ECU) connections.",
'ag_003':  "Inspect  brake  fluid  and  hydraulic  line  pressure.",
'ba_003': "Verify electrical wiring  for wear and short circuits.",
'cn_000': "Inspect and clean air filters in the control system.",
'cs_004':  "Check  for  any  malfunction  in  the  signal  processing
system."  }
    likely_faulty_features = {feature: solutions[feature] for
feature in top_features['feature'] if feature in solutions}

    data = {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f2_score": f2,
        "likely_faulty_features":  likely_faulty_features
    }
    return render(request, 'user/Gboost_prediction.html', data)
  return  render(request,  'user/Gboost_prediction.html')
```

CHAPTER-6

TESTING

CHAPTER-6

TESTING

Purpose of Testing

The main goal of software testing in this project is to identify errors and ensure that the brake fault prediction system functions correctly. Testing verifies that the system accurately processes sensor data and produces correct predictions using machine learning models. It also ensures that Explainable AI techniques like SHAP and LIME provide clear and understandable explanations.

Testing improves the reliability, performance, security, and usability of the system in real-world conditions before deployment.

Key Testing Methodologies (Software Testing Types)

Different testing approaches are used at various stages of development to ensure the system works efficiently and accurately. Below are the commonly used software testing methodologies:

Functional Testing -

There are four main types of functional testing.

1. **Unit Testing:** Unit testing is used to verify that individual components or modules of the brake fault prediction system work correctly in isolation.

   Example:

   In this system, a module handles data preprocessing, such as cleaning sensor data, handling missing values, and normalization before feeding it into the machine learning models.

   - **Test Objective:** Ensure that preprocessing functions correctly transform input data without errors and maintain data consistency for accurate model training

2. **Integration Testing:** Integration testing ensures that different modules of the system work together properly.

   Example:

   The system consists of multiple components such as:

* Data Upload Module

* Data Preprocessing Module

* Machine Learning Model

* Prediction Module

* Explainability Module (SHAP/LIME)

- **Test Objective:** Verify that all modules interact correctly and data flows properly between them to produce accurate brake fault predictions.

3. **User Acceptance Testing (UAT):** User Acceptance Testing ensures that the system meets user expectations and works effectively in real-world scenarios.

   Example:

   Users provide sensor input values, and the system predicts whether a brake fault exists or not, along with explanation results.

   - **Test Objective:** Ensure that predictions are accurate, meaningful, and useful for users like vehicle operators or engineers.

➢ **Accuracy Test:** Are the predictions correct based on input data? Are SHAP/LIME explanations understandable to users?

4. **Output Testing:** Output testing ensures that the results generated by the system are accurate, well-structured, and easy to understand.

   Example:

   The system displays:

   * Fault / No Fault result

   * Accuracy metrics

   * Feature importance (SHAP/LIME graphs)

   - **Test Objective:** Ensure that output is correctly displayed and provides useful insights to the user.

➢ **Accuracy Test:** Is the prediction result correct? Are explanation graphs clearly visible and meaningful?

5. **Validation Testing:** Validation testing ensures that the system handles invalid or unexpected inputs properly without crashing.

   Example:

   Users may:

   * Upload incorrect file formats

   * Enter invalid sensor values

   * Provide empty inputs

- **Test Objective:** Ensure that the system validates all inputs and handles errors properly by displaying meaningful error messages instead of crashing.

1. UNIT TEST CASES

6.1.1 Data Upload Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT1	Upload valid dataset	CSV file	File uploaded successfully	As expected	Pass
UT2	Upload invalid file format	.txt file	Error message displayed	As expected	Pass
UT3	Upload empty file	Empty CSV	Error message	As expected	Pass
UT4	Upload large dataset	Large CSV	File uploaded without crash	As expected	Pass

6.1.2 Data Preprocessing Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT5	Handle missing values	Dataset with null values	Missing values handled	As expected	Pass
UT6	Normalize data	Raw dataset	Scaled dataset	As expected	Pass
UT7	Invalid data types	Dataset with strings	Error/Handled properly	As expected	Pass

6.1.3 Model Training Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT8	Train model with valid data	Processed dataset	Processed dataset	As expected	Pass
UT9	Train with insufficient data	Small dataset	Warning/Error	As expected	Pass
UT10	Multiple models training	Dataset	Best model selected	As expected	Pass

6.1.4 Prediction Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT11	Predict fault	Valid input values	Fault/No Fault result	As expected	Pass
UT12	Invalid input values	Wrong format	Error message	As expected	Pass
UT13	Boundary values	Edge inputs	Correct prediction	As expected	Pass

6.1.5 Explainability Module (XAI)

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT14	Generate explanation	Prediction result	Feature importance shown	As expected	Pass
UT15	Invalid model	No model	Error message	As expected	Pass

6.1.6 User Interface Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
UT16	Load user page	Open UI	Page loads correctly	As expected	Pass
UT17	Submit form	Input values	Results displayed	As expected	Pass
UT18	Invalid inputs	Wrong data	Error shown	As expected	Pass

2. BLACK BOX TEST CASES

6.2.1 Boundary Value Testing

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
BB1	Minimum input values	Lower limit values	Valid output	As expected	Pass
BB2	Maximum input values	Upper limit values	Valid output	As expected	Pass

6.2.2 Functional Testing (Black Box)

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
BB3	Upload dataset	Valid CSV	File uploaded	As expected	Pass
BB4	Train model	Dataset	Model trained	As expected	Pass
BB5	Predict brake fault	Sensor values	Fault/No Fault	As expected	Pass
BB6	View explanation	Prediction result	Explanation displayed	As expected	Pass
BB7	Invalid dataset	Wrong file	Error message	As expected	Pass

6.2.3 Error Handling Testing

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
BB8	Empty input fields	No input	Error message	As expected	Pass
BB9	Invalid format	Text instead of numbers	Error message	As expected	Pass

6.2.4 Performance Testing

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
BB10	Large dataset processing	Big dataset	System handles efficiently	As expected	Pass
BB11	Multiple users	Concurrent access	No crash	As expected	Pass

6.2.5 Security Testing

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
BB12	Unauthorized access	Without login	Access denied	As expected	Pass
BB13	Data privacy	Sensitive data	Data secured	As expected	Pass

CHAPTER-7

RESULTS

CHAPTER-7
RESULTS

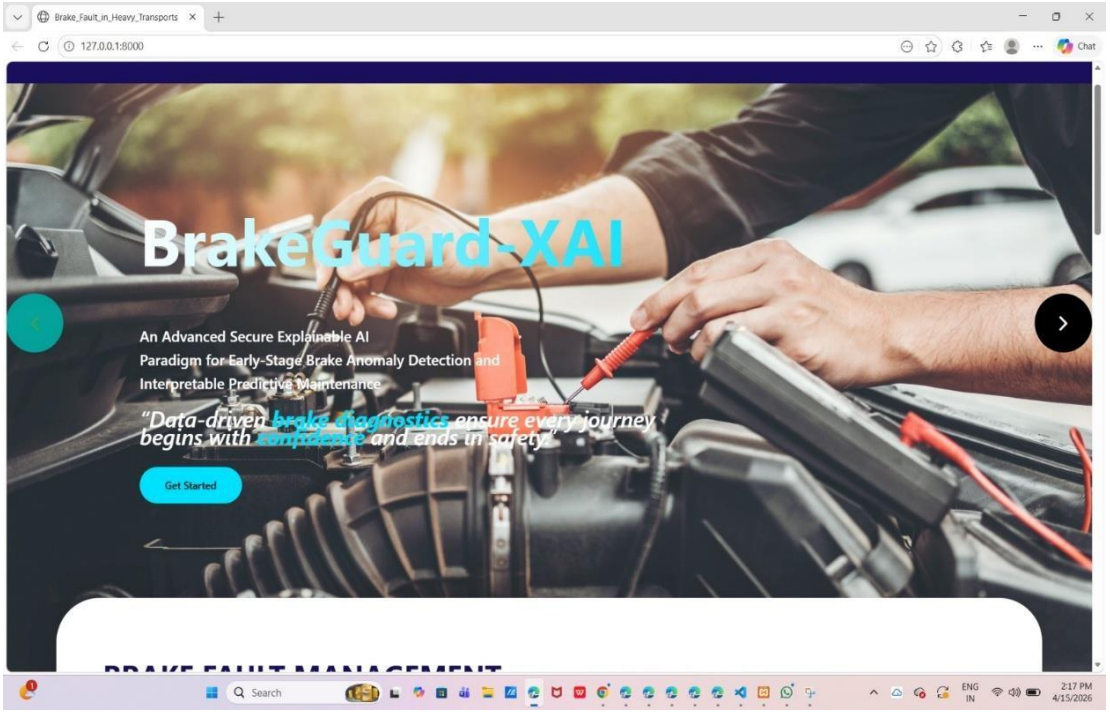

Fig. 7.1 Home Screen
Shows the system home interface with navigation options.

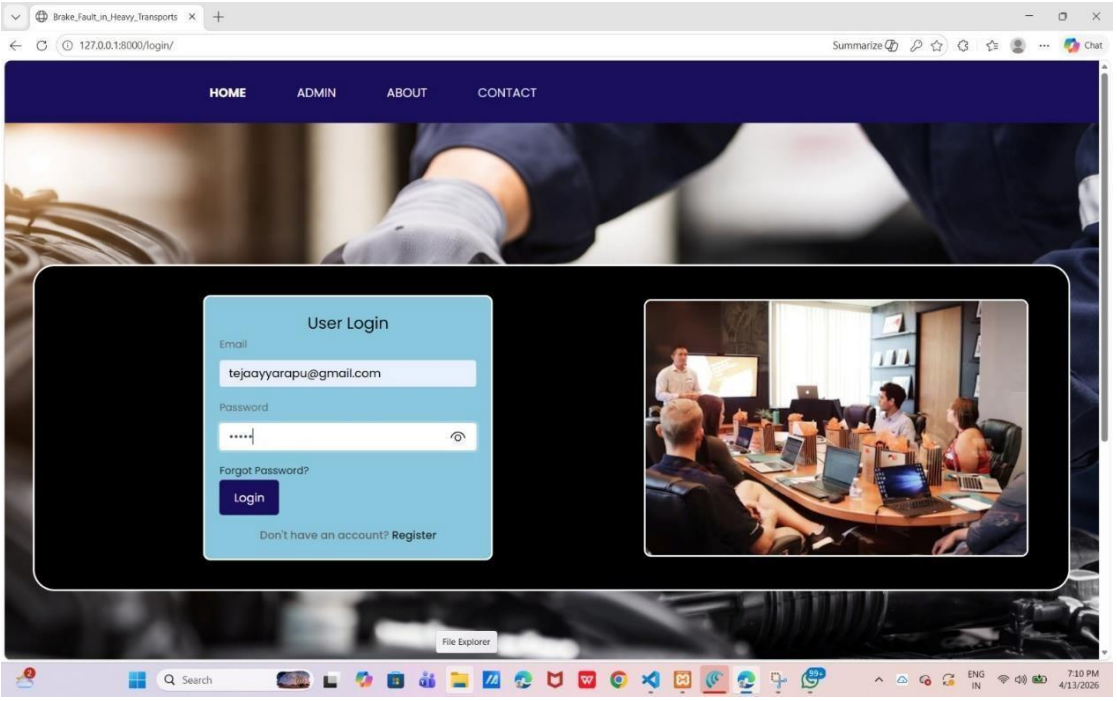

Fig. 7.2 Remote user login Page
Shows the login page for registered users to access the system.

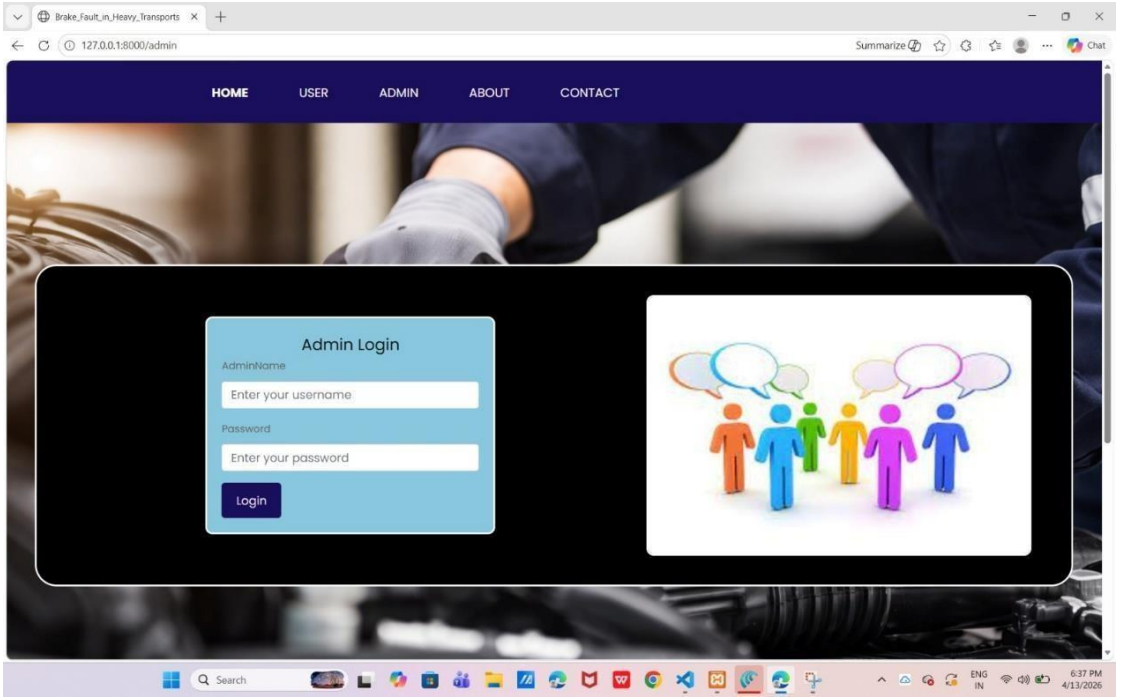

Fig. 7.3 Admin login page
Displays the admin login page for secure access.

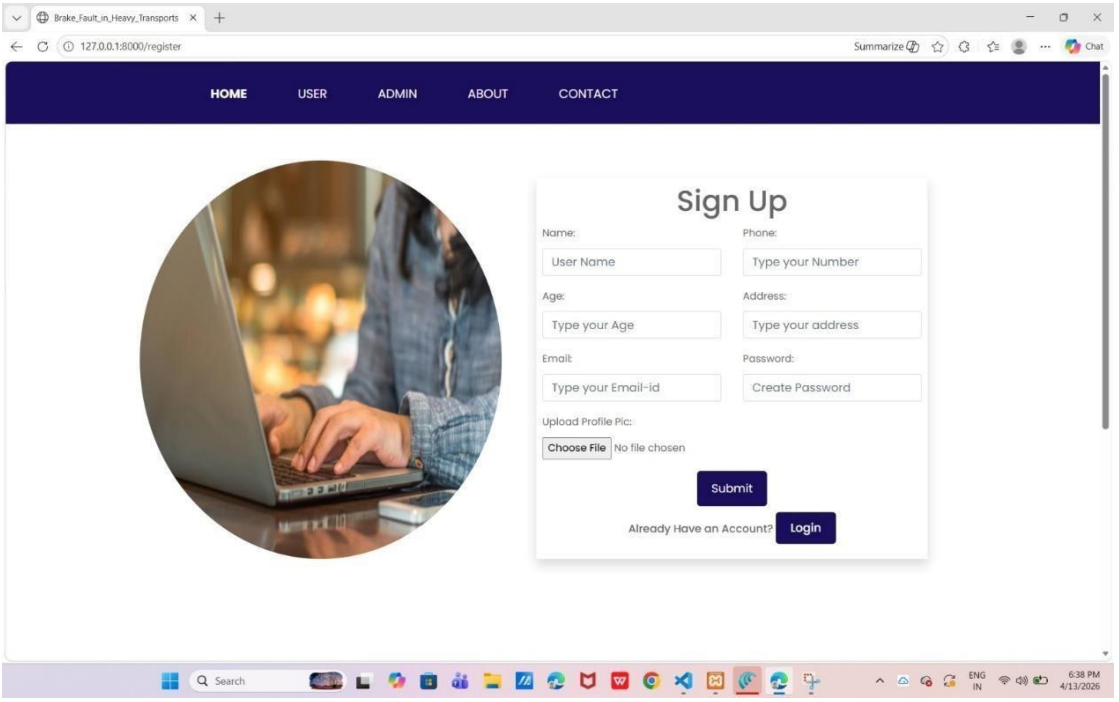

Fig. 7.4 User Registration Form
Displays the user registration form for new users.

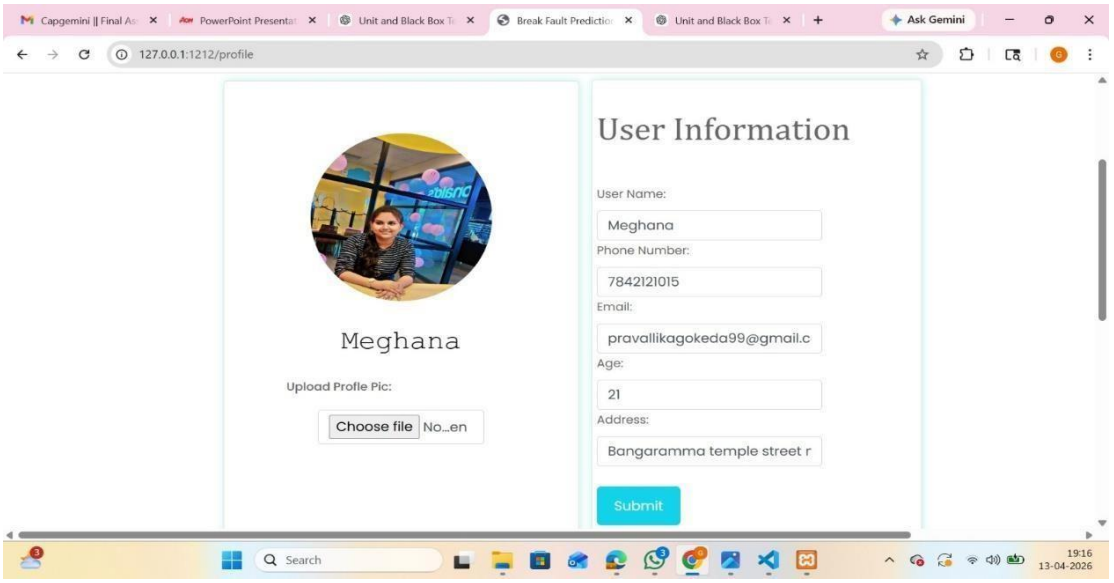

Fig. 7.5 Remote user Profile

Shows user profile details and account information.

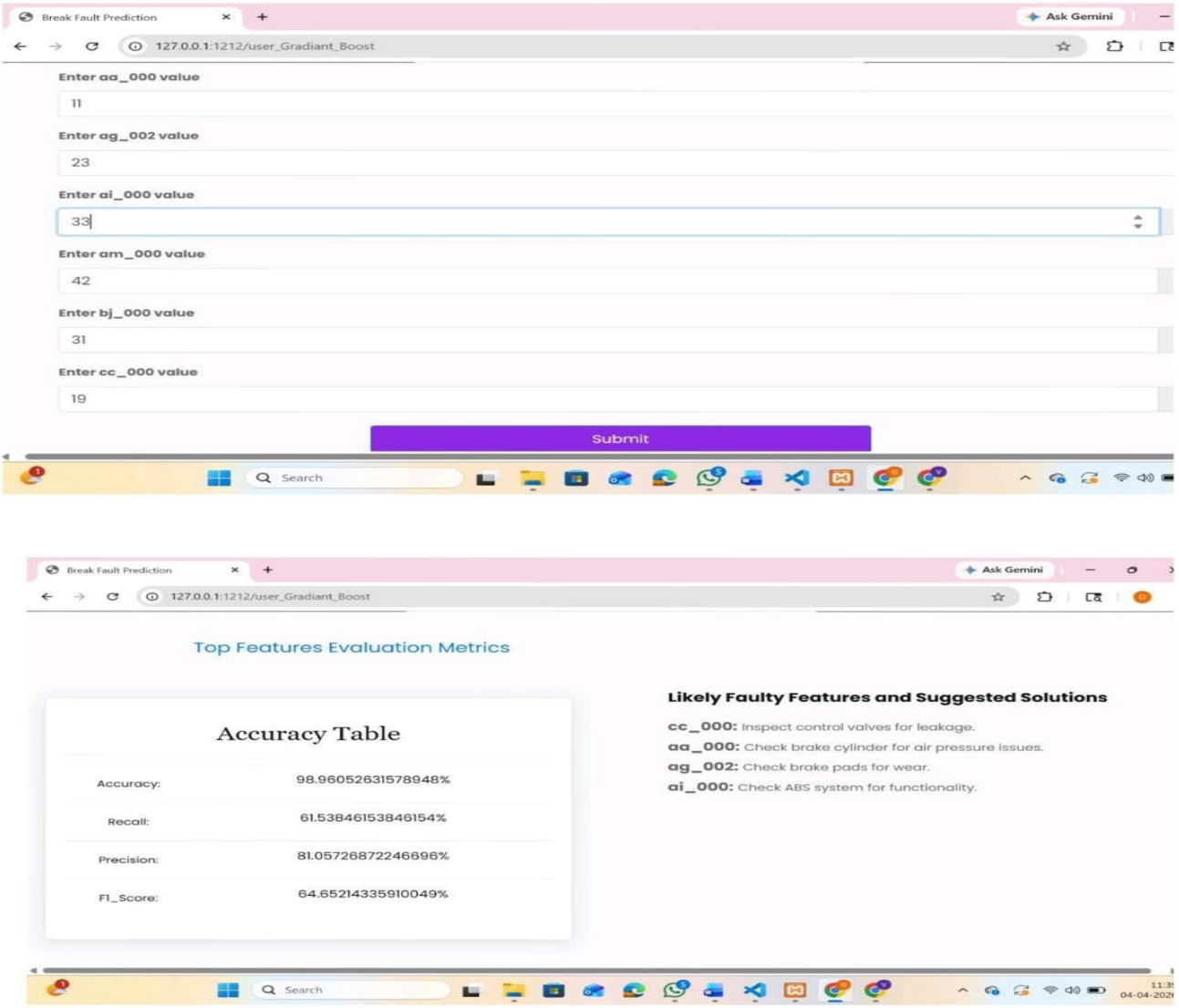

Fig. 7.6 Predicting Software Defect

Shows input values and displays brake fault prediction results.

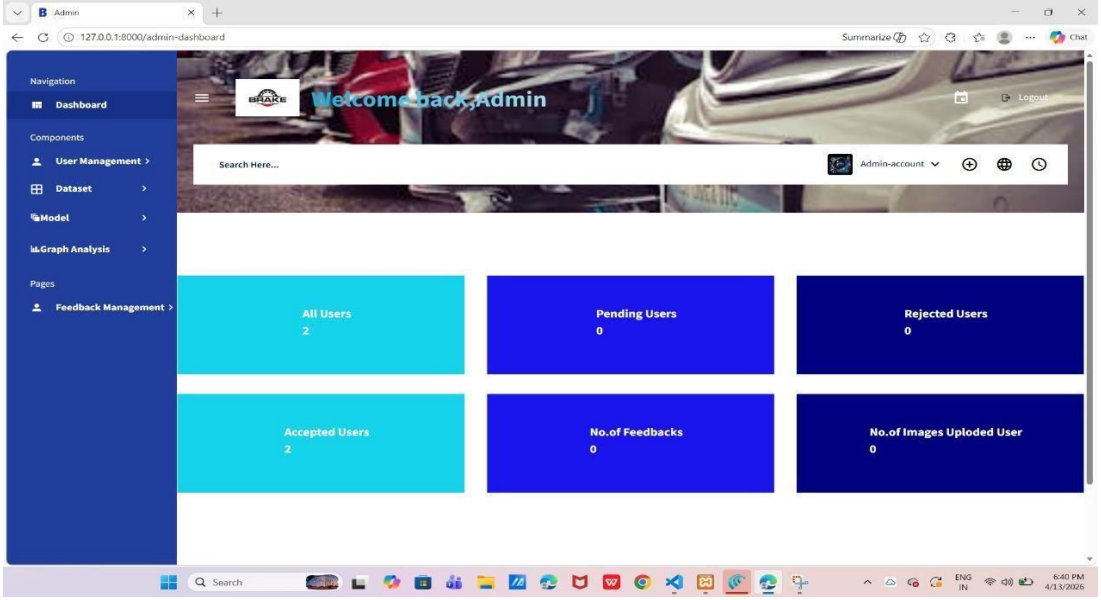

Fig. 7.7 Admin dashboard

Displays the admin dashboard with user statistics and system activity overview.

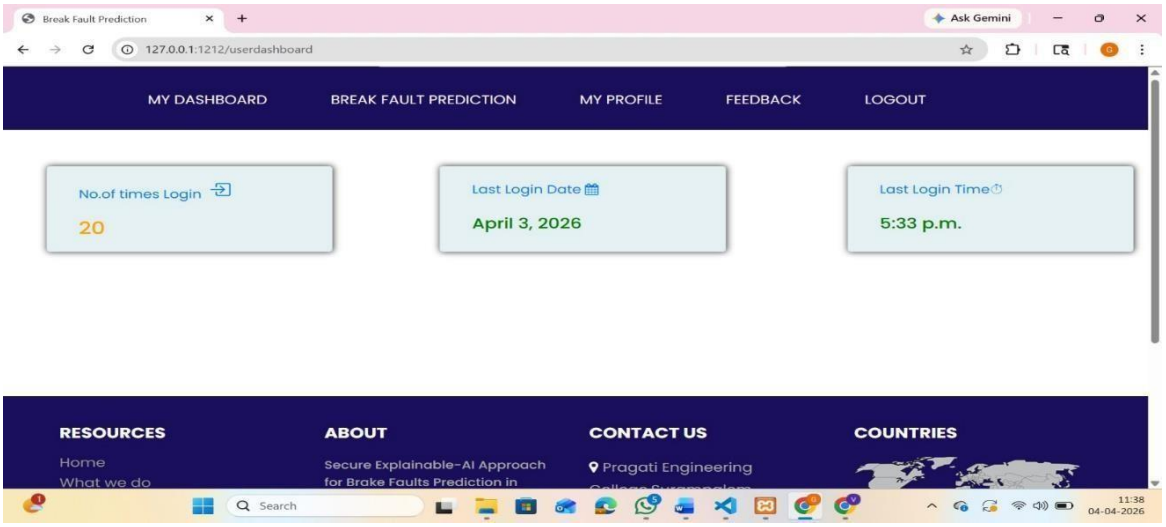

Fig. 7.8 User dashboard

Displays user activity details such as login information.

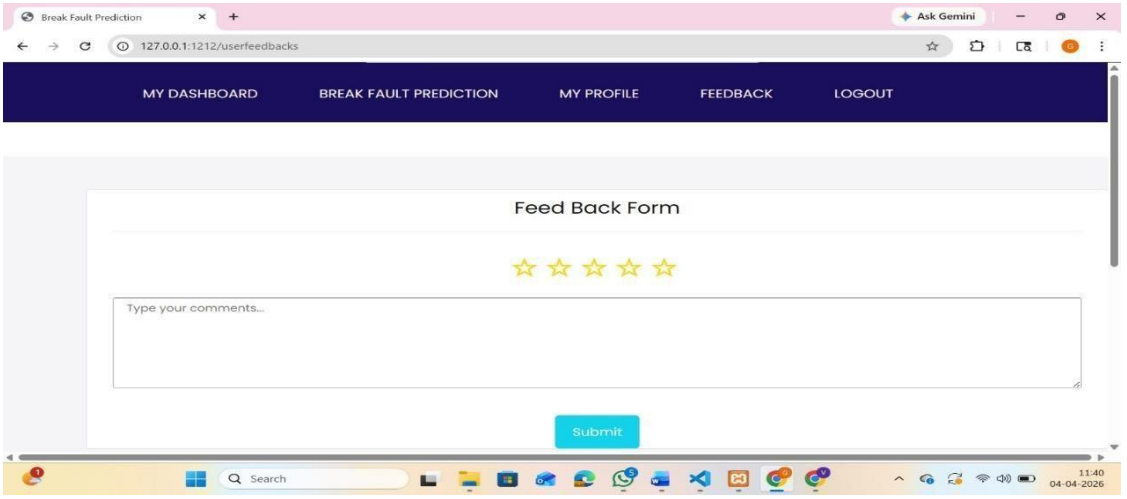

Fig. 7.9 User feedback form

Shows the feedback form for users to submit their responses.

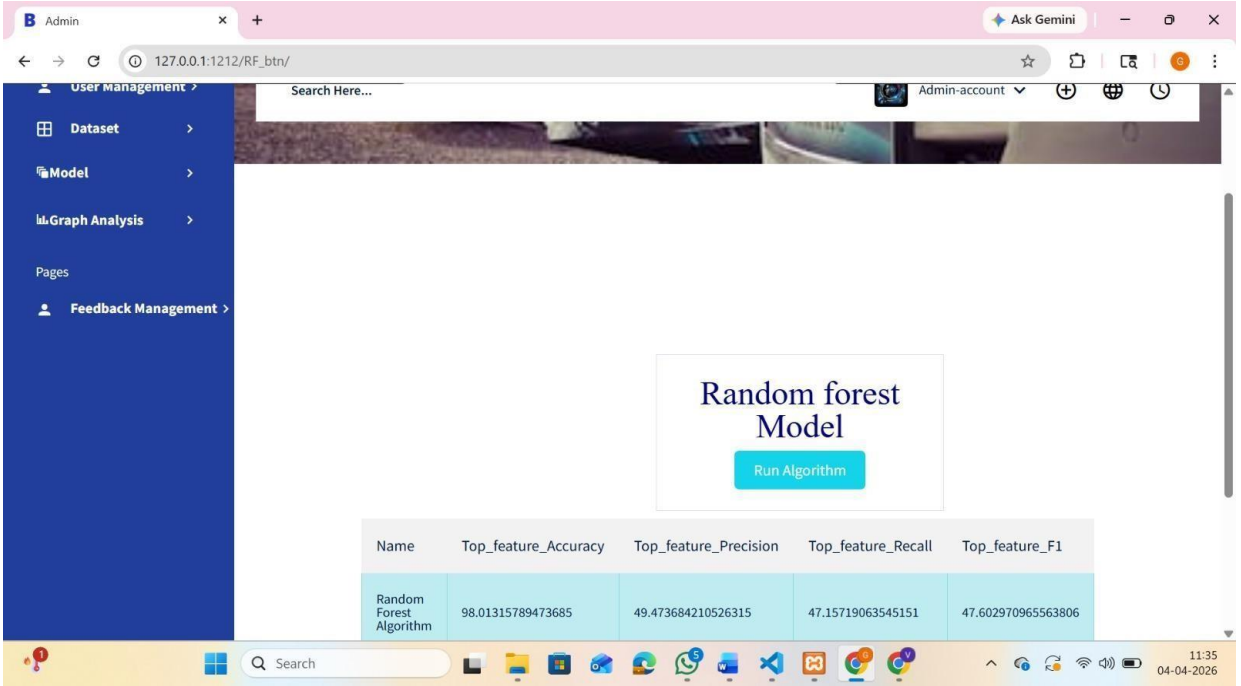

Fig. 7.10 Random forest model execution dashboard
Shows Random Forest model evaluation results.

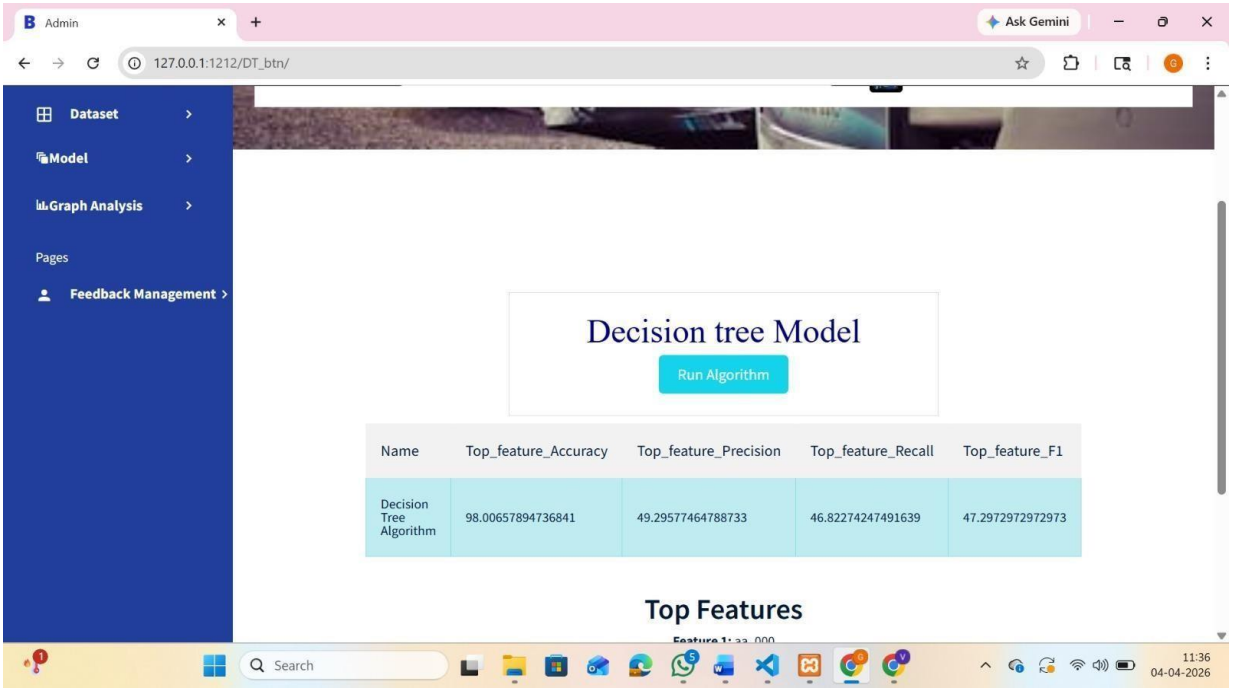

Fig. 7.11 Decision tree model execution dashboard
Displays Decision Tree model performance metrics.

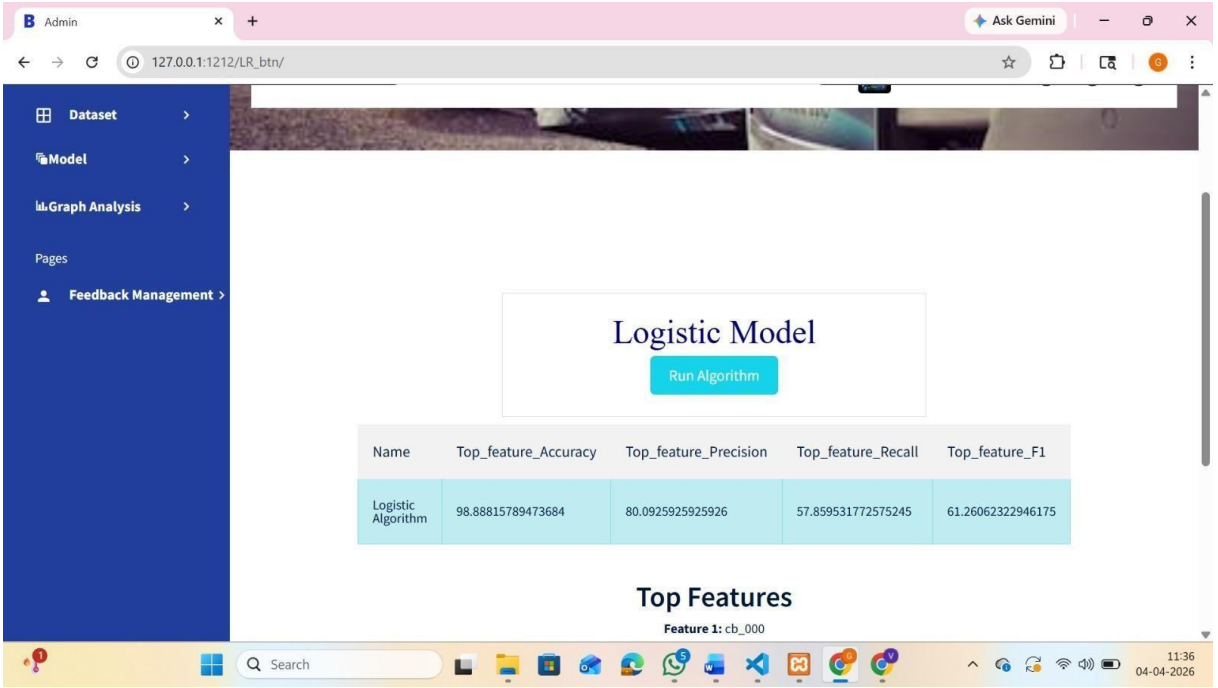

Fig. 7.12 Logistic model execution dashboard
Displays Logistic Regression model evaluation results.

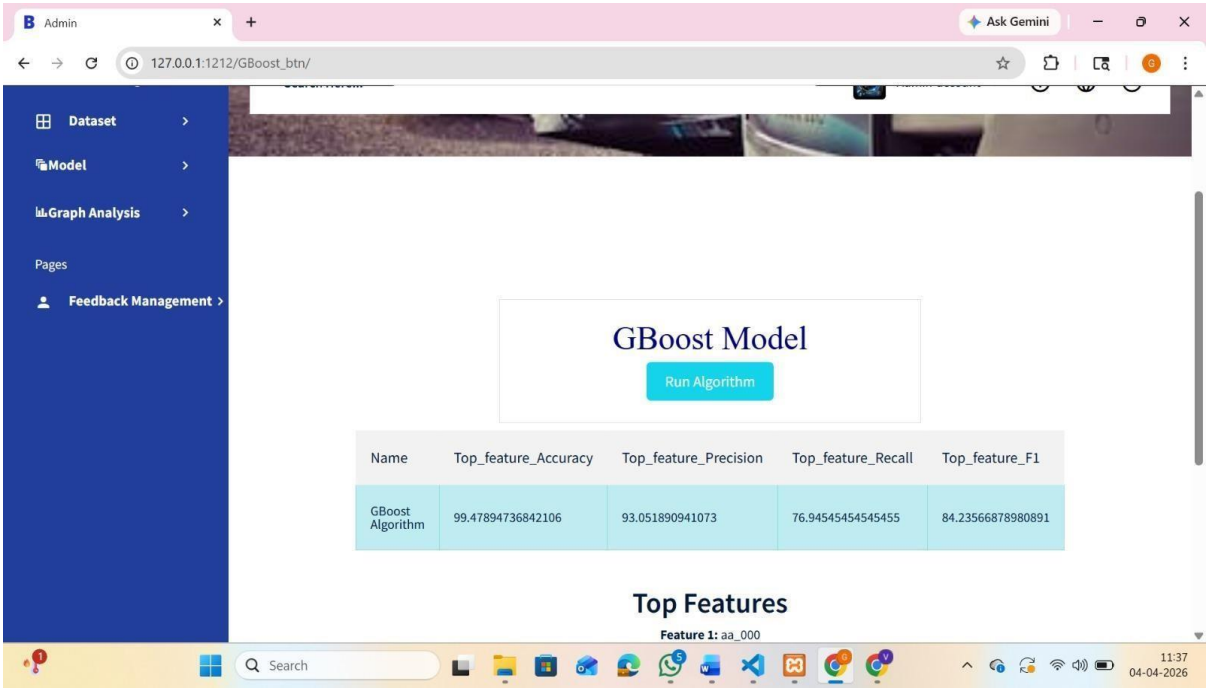

Fig. 7.13 GBoost model execution dashboard
Shows Gradient Boosting model performance metrics.

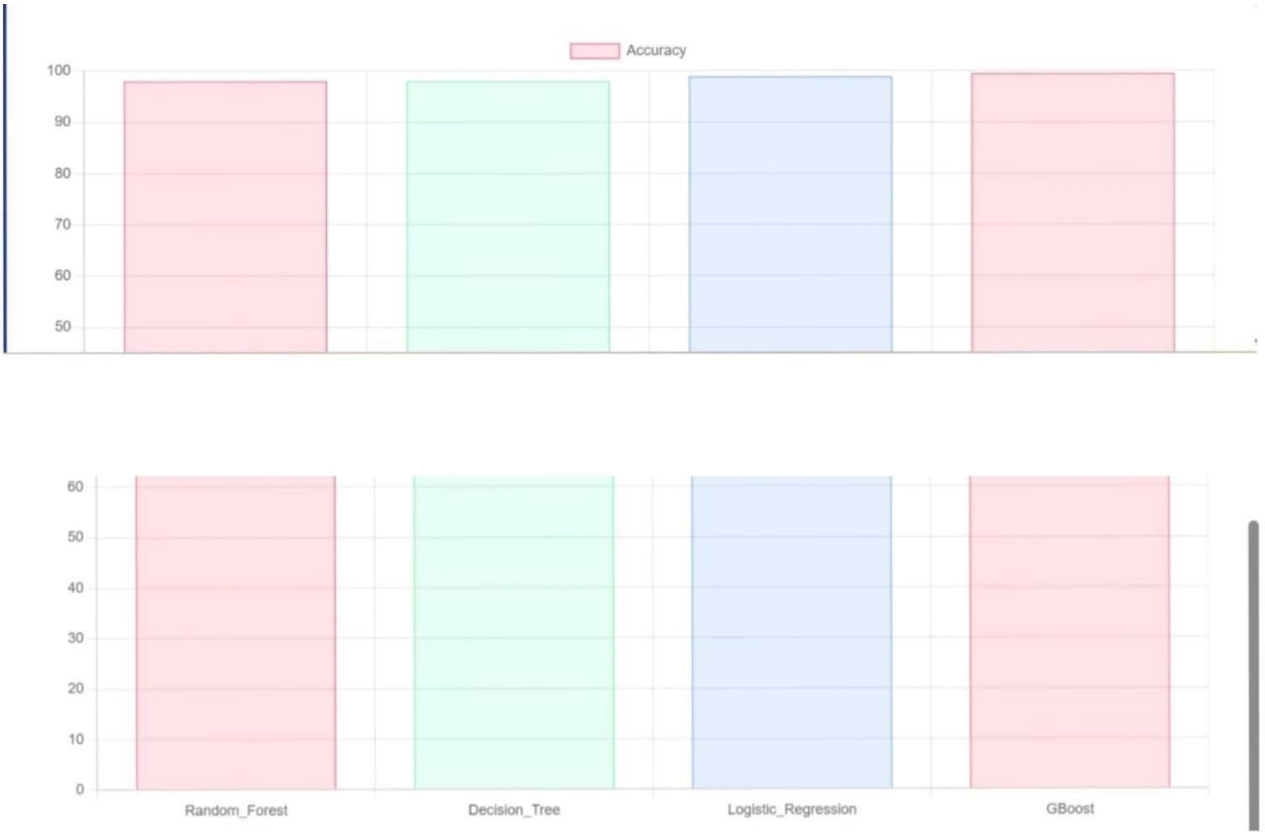

Fig. 7.14 Model accuracy graph
Compares accuracy of different machine learning models.

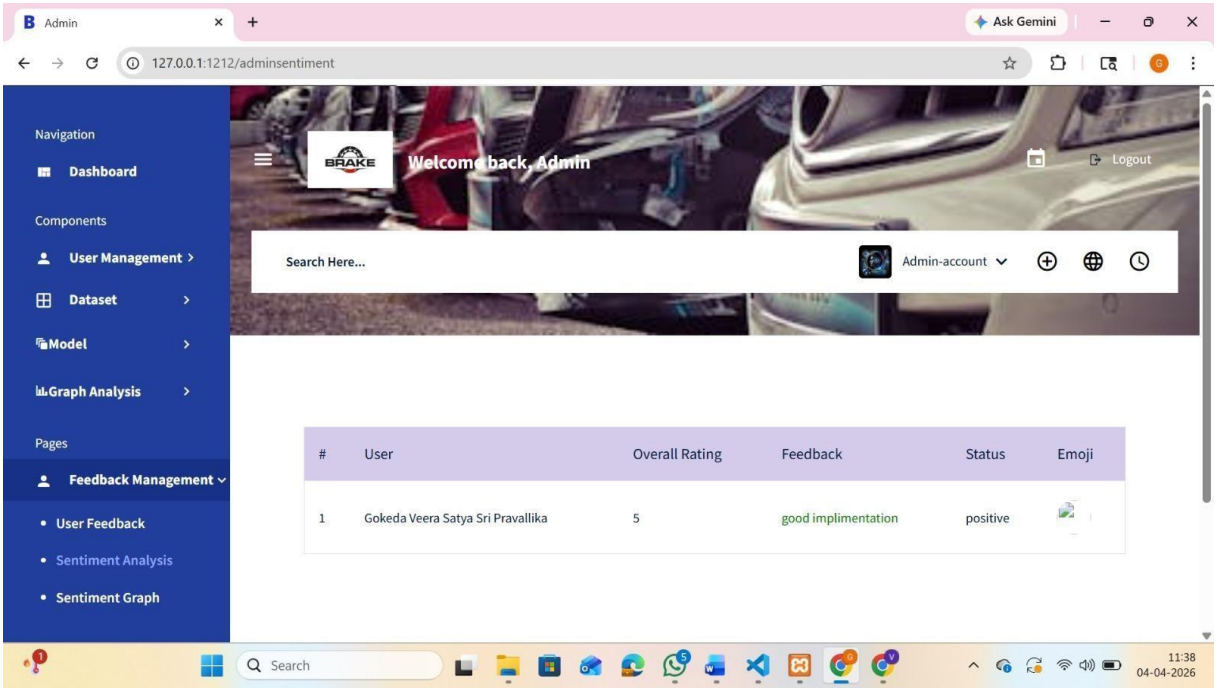

Fig. 7.15 Feedback analysis
Displays user feedback with sentiment analysis.

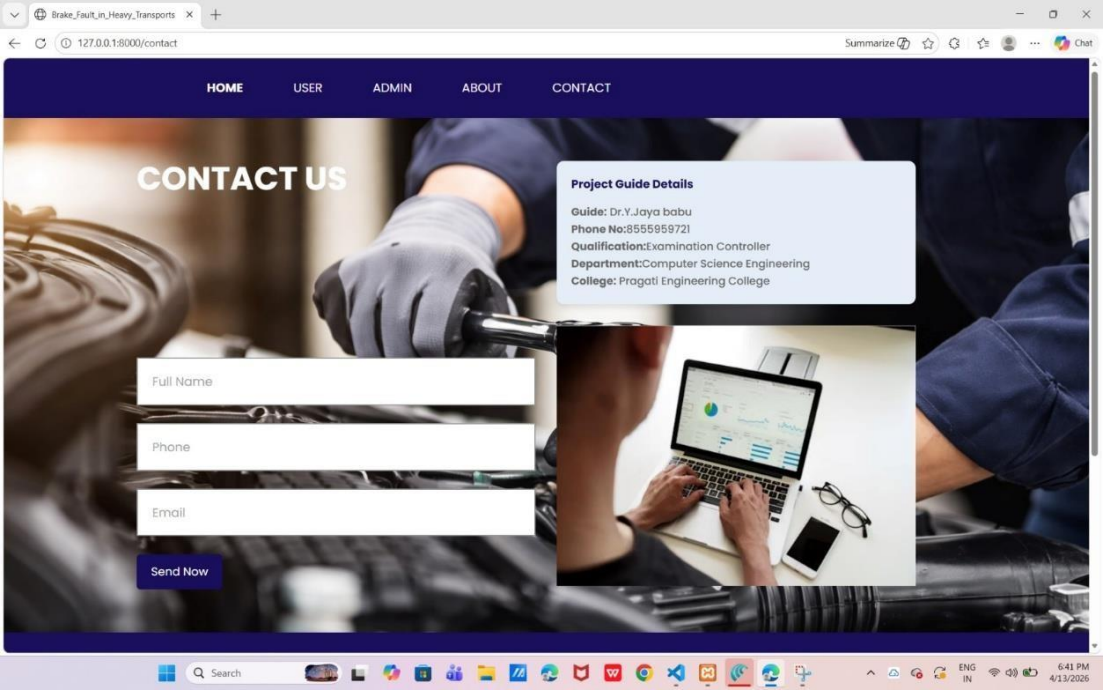

Fig. 7.16 Contact us form

Provides contact details and communication form.

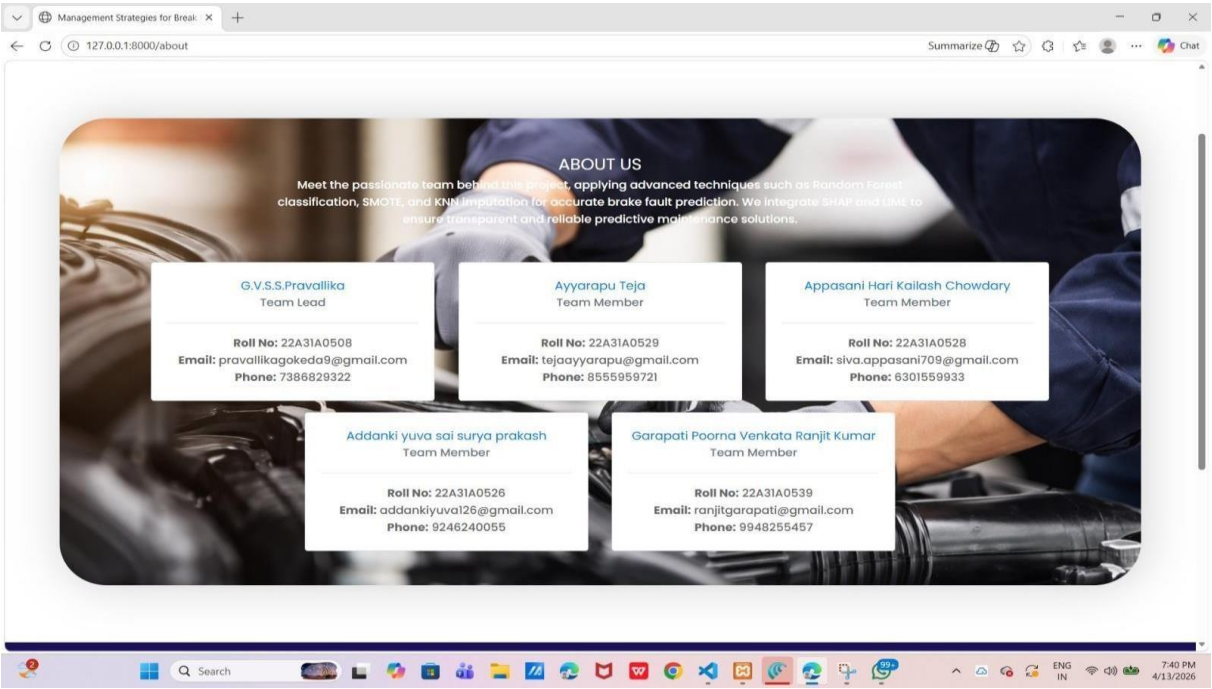

Fig. 7.17 About Us

Shows project and team information.

CHAPTER-8

CONCLUSION AND FUTURE WORK

CHAPTER-8

CONCLUSION AND FUTURE WORK

This project presented a secure and explainable AI-based approach for predicting brake faults in heavy transport vehicles. The main focus was to detect failures in the air pressure system using sensor data collected from heavy-duty vehicles. Machine learning models were applied for fault classification, and Random Forest was identified as the best-performing model because it achieved very high accuracy and stable results. This helps in detecting brake faults early, which improves vehicle safety and reduces unexpected breakdowns.

To improve transparency and trust, Explainable AI techniques such as SHAP and LIME were integrated into the system. These techniques clearly explain how different sensor features influence the final prediction, which makes the model more understandable for maintenance engineers. The system also reduced complexity by selecting only the most important features from the full dataset. This reduced the computational cost and made the solution more efficient, while still maintaining the same level of prediction accuracy.

Overall, the proposed approach improves predictive maintenance, supports safer heavy transport operations, and provides a reliable AI-based solution that is both accurate and explainable.

This project focused on developing an intelligent system for early detection of brake faults using machine learning and Explainable AI. It involves data preprocessing, feature selection, model training, and evaluation using multiple classification algorithms. Random Forest was identified as the most effective model for fault prediction. Explainability methods like SHAP and LIME were applied to interpret model decisions and improve trust. The study emphasizes the importance of combining accuracy, interpretability, and security to build reliable AI-based solutions for transportation systems.

Future Work

In future work, the system can be enhanced by using advanced explainability methods such as DeepSHAP and DeepLIFT. These methods can provide deeper and more detailed explanations, especially when more complex models are used. The system can also be improved by integrating real-time IoT sensor streaming. This will allow brake faults to be detected instantly during vehicle operation, making the system more practical for real-world deployment.

Another improvement is to test the model on larger datasets and on different types of heavy vehicles. This will help the system become more general and reliable across different driving conditions and vehicle designs. Security and privacy can be strengthened further by applying stronger privacy-preserving techniques to protect sensitive vehicle sensor data during storage, processing, and transmission.

Finally, the proposed model can be extended into a complete intelligent maintenance platform. Such a platform can provide early warning alerts, fault severity levels, and maintenance recommendations, which will help transport companies reduce cost and improve safety.

BIBLIOGRAPHY

1. S. Nazat, L. Li, and M. Abdallah, ‘‘XAI-ADS: An explainable artificial intelligence framework for enhancing anomaly detection in autonomous driving systems,’’ IEEE Access, vol. 12, pp. 48583–48607, 2024.
2. C. N. Spartalis, T. Semertzidis, and P. Daras, ‘‘Balancing XAI with privacy and security considerations,’’ in Proc. Eur. Symp. Res. Comput. Secur.,2023, pp. 111–12.
3. R. Raveendran, K. Devika, and S. C. Subramanian, ‘‘Learning-based fault diagnosis of air brake system using wheel speed data,’’ Proc. Inst. Mech. Engineers, Part D, J. Automobile Eng., vol. 236, no. 12, pp. 2598–2609,Oct. 2022.
4. I. C. Guleryuz and Ö. Baser, ‘‘Modelling the longitudinal braking dynamics for heavy-duty vehicles,’’ Proc. Inst. Mech. Engineers,Part D, J. Automobile Eng., vol. 235, nos. 10–11, pp. 2802–2817,Sep. 2021.
5. S. Matzka, ‘‘Explainable AI for predictive maintenance applications,’’ in Proc. 3rd Int. Conf. Artif. Intell. Industries (AI4I),Sep. 2020, pp. 69–74.
6. A. Barredo Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, ‘‘Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,’’ Inf. Fusion, vol. 58, pp. 82–115, Jun. 2020.
7. A. Barredo Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, ‘‘Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,’’ Inf. Fusion, vol. 58, pp. 82–115, Jun. 2020.
8. A. Badshah, A. Ghani, A. Daud, A. Jalal, M. Bilal, and J. Crowcroft, ‘‘Towards smart education through Internet of Things: A survey,’’ ACM Comput. Surveys, vol. 56, no. 2, pp. 1–33, Feb. 2024.
9. V.Balasubramaniam,‘‘Artificial intelligence algorithm with SVM classifi cation using dermascopic images for melanoma diagnosis,’’ J. Artif. Intell. Capsule Netw., vol. 3, no. 1, pp. 34–42, Mar. 2021.
10. C. Singh and W. Lin, ‘‘Can artificial intelligence, RegTech and Chari tyTech provide effective solutions for anti-money laundering and counter terror financing initiatives in charitable fundraising,’’ J. Money Laundering Control, vol. 24, no. 3, pp. 464–482, Jul. 2021.

11. N. A. Hamad, K. M. A. Alheeti, and S. S. Al-Rawi, ‘‘Intrusion detection system using artificial intelligence for internal messages of robotic cars,’’ in AIP Conf. Proc., 2022, pp. 1–20.
12. X. Wang and M. Yin, ‘‘Are explanations helpful? A comparative study of the effects of explanations in AI-assisted decision-making,’’ in Proc. 26th Int. Conf. Intell. User Inter., Apr. 2021, pp. 318–328.
13. R. Hamon, H. Junklewitz, I. Sanchez, G. Malgieri, and P. De Hert, ‘‘Bridging the gap between AI and explainability in the GDPR: Towards trustworthiness-by-design in automated decision-making,’’ IEEE Comput. Intell. Mag., vol. 17, no. 1, pp. 72–85, Feb. 2022.
14. Z. Miftioglu, M. A. Kızrak, and T. Yıldırım, ‘‘Privacy-preserving mechanisms with explainability in assistive AI technologies,’’ in Learning and Analytics in Intelligent Systems. Cham, Switzerland: Springer, 2022, pp. 287–309.
15. I. A. K. Soudagar and P. D. Tota, ‘‘Modelling and simulation of electro pneumatic parking brake system for real time estimation of pressure inside parking brake chamber,’’ Tech. Rep., 2022.
16. J. S. Avliyokulov, M. S. Pulatovich, and M. I. Rakhmatov, ‘‘Main failures of the vehicle brake system, maintenance and repair,’’ Cen tral ASIAN J. Math. Theory Comput. Sci., vol. 4, no. 3, pp. 63–69, 2023.
17. X. Hua, J. Zeng, H. Li, J. Huang, M. Luo, X. Feng, H. Xiong, and W. Wu, ‘‘A review of automobile brake-by-wire control technology,’’ Processes, vol. 11, no. 4, p. 994, Mar. 2023.
18. M. H. Abidi, M. K. Mohammed, and H. Alkhalefah, ‘‘Predictive maintenance planning for industry 4.0 using machine learning for sustainable manufacturing,’’ Sustainability, vol. 14, no. 6, p. 3387, Mar. 2022.
19. N. Omrani, G. Rivieccio, U. Fiore, F. Schiavone, and S. G. Agreda, ‘‘To trust or not to trust? An assessment of trust in AI-based systems: Concerns, ethics and contexts,’’ Technological Forecasting Social Change, vol. 181, Aug. 2022, Art. no. 121763.
20. R. Alharbey, A. Shafiq, A. Daud, H. Dawood, A. Bukhari, and B. Alshemaimri, ‘‘Digital twin technology for enhanced smart grid performance: Integrating sustainability, security, and efficiency,’’ Frontiers Energy Res., vol. 12, Jun. 2024, Art. no. 1397748.

Paper Publication

International Journal of Scientific Research & Engineering Trends
Volume 12, Issue 2, Mar-Apr-2026, ISSN (Online): 2395-566X

BrakeGuard-XAI – An Advanced Secure Explainable AI Paradigm for Early-Stage Brake Anomaly Detection and Interpretable Predictive Maintenance

Dr. Y. Jayababu[1] , Gokeda Veera Satya Sri Pravallika[2], Ayyarapu Teja[3], Appasani Hari Kailash Chowdary[4], Addanki Yuva Sai Surya Prakash[5], Garapati Poorna Venkata Ranjit Kumar[6]
[1]Professor, [2, 3,4,5,6] B.tech Students Department of CSE, Pragati Engineering College,
Surampalem , Andhra Pradesh, India

Abstract- The study suggests an accessible and secure machine learning model for forecasting brake failures in large commercial vehicles. We support this proposal with evidence. Heavy transport vehicles' Air Pressure System (APS) is constantly monitored by IoT-based sensors in modern day heavy transport systems, generating vast amounts of operational data. Detecting brake failures manually with large and highly unbalanced datasets is time-consuming and inefficient. Our approach to these problems involves the use of K-Nearest Neighbour (KNN) imputation for missing values and SMOTE for dealing with class imbalance. Both methods are effective in both situations. Logistic Regression, Decision Tree, Support Vector Machine, Gradient Boosting, and Random Forest are among the machine learning algorithms that undergo stratified cross-validation during implementation and evaluation. The Random Forest classifier's accuracy, precision, recall, F1-score and ROC-AUC are shown to be more than satisfactory using experimental data. Enhanced transparency and trust in the prediction process are achieved through the use of Explainable Artificial Intelligence (XAI) techniques like SHAP and LIME, which can interpret model decisions. They also use methods of selecting features that reduce computational complexity while preserving high levels of accuracy in making predictions. This proposed framework improves fault detection reliability, reduces maintenance costs and allows for predictive maintenance in heavy transport systems.

Keywords – Brake Fault Prediction, Random Forest, Explainable AI, SHAP, LIME, SMOTE, Predictive Maintenance, Machine Learning, Imbalanced Data.

I. INTRODUCTION

The rapid expansion of commercial transportation has significantly increased the demand for safe, reliable, and intelligent vehicle operation. Heavy commercial vehicles depend heavily on efficient braking systems to ensure passenger safety, cargo protection, and accident prevention. Any malfunction in the braking mechanism can lead to severe operational risks and financial losses.

In particular, the Air Pressure System (APS), which controls the braking process in heavy vehicles, plays a crucial role in maintaining vehicle safety. Failures within the APS may result in compromised braking performance, thereby increasing the likelihood of accidents. Consequently, early detection and prediction of brake faults have become essential components of modern intelligent transportation and predictive maintenance systems.

Traditionally, brake fault detection relies on periodic manual inspections and rule-based diagnostic procedures. While these approaches can detect certain mechanical issues, they are often time-consuming, labour-intensive, and inefficient when dealing with large-scale sensor data generated by modern vehicles. Contemporary heavy vehicles are equipped with numerous IoT-enabled sensors that continuously monitor various operational parameters. These sensors generate substantial volumes of real-time data, enabling advanced monitoring and analysis of vehicle performance. However, the high dimensionality and complexity of such datasets make manual analysis impractical, thereby necessitating the use of data-driven analytical methods for effective fault detection and predictive maintenance [3], [4].

Machine learning (ML) techniques have emerged as powerful tools for predictive maintenance and fault diagnosis in intelligent transportation systems. By learning patterns from historical vehicle operational data, ML models can effectively differentiate between normal and abnormal system behaviours, thereby facilitating early detection of potential failures. These intelligent techniques have been successfully applied across multiple domains, including healthcare

© 2026 IJSRET

diagnostics, anomaly detection, and autonomous systems, demonstrating their capability to identify complex patterns within large and heterogeneous datasets [5], [7], [13].

Despite their advantages, real-world industrial datasets present several significant challenges. These include missing values, high-dimensional feature spaces, and severe class imbalance, where the number of failure instances is considerably smaller than the number of normal operational samples.

Such data characteristics can negatively affect model training and predictive performance if not properly addressed. In addition, many high-performing machine learning algorithms operate as black-box models, making it difficult to interpret the reasoning behind their predictions. This lack of transparency is particularly problematic in safety-critical applications such as brake fault prediction, where explain ability and trust are essential for reliable decision-making [8], [11].

To address these limitations, Explainable Artificial Intelligence (XAI) has emerged as an important research direction aimed at improving the interpretability and transparency of machine learning models. XAI techniques provide insights into model predictions by identifying the most influential features and explaining the reasoning behind classification decisions. Such interpretability mechanisms are essential for enhancing user trust, improving system reliability, and supporting regulatory compliance in AI-driven decision-making systems [1], [2], [9], [10], [12].

Motivated by these challenges, this paper proposes a secure and explainable machine learning framework for predicting brake faults in heavy commercial vehicles. The proposed framework incorporates robust data preprocessing techniques to address missing values and class imbalance, evaluates multiple classification algorithms for fault prediction, and integrates explainability techniques such as SHAP and LIME to improve model transparency.

The objective of the proposed system is to achieve high predictive accuracy while maintaining interpretability and computational efficiency, thereby supporting trustworthy predictive maintenance in intelligent transportation systems.

The remainder of this paper is organized as follows. Section II reviews existing studies related to brake fault detection and anomaly prediction techniques. Section III presents the analysis of the existing system and the proposed approach. Section IV describes the system architecture and design methodology. Section V outlines the implementation modules. Section VI discusses the experimental results and performance evaluation. Finally, Section VII summarizes the contributions and concludes the paper.

II. LITERATURE SURVEY

Many researchers have applied machine learning and data-driven approaches to identify hidden patterns across various industrial domains, including transportation systems, where such techniques are widely used for predictive maintenance and fault diagnosis. With the rapid development of sensor-based monitoring systems and IoT-enabled infrastructures, intelligent fault diagnosis has become a major research focus in modern industrial and transportation systems [3], [4].

Halvaiee and Akbari proposed the use of an Artificial Immune System for fraud detection with the objective of improving classification accuracy and reducing system response time. Their study demonstrated that biologically inspired algorithms can enhance detection efficiency compared with traditional classification approaches. Although the proposed method achieved improved performance, it primarily focused on financial datasets and lacked interpretability, which limits its applicability in safety-critical domains where model transparency is essential.

To improve the effectiveness of detection-based systems, Bahnsen et al. introduced several feature engineering techniques designed to enhance classification performance. Their approach involved generating additional statistical features from temporal transaction patterns to improve predictive capability. The study highlighted the importance of feature extraction in achieving higher predictive accuracy, particularly in high-dimensional datasets. Multiple machine learning algorithms, including Naïve Bayes, Decision Trees, Support Vector Machines, and Neural Networks, were evaluated to assess the effectiveness of the engineered features.

Furthermore, Randhawa et al. investigated ensemble learning approaches such as AdaBoost combined with majority voting strategies. Their comparative analysis of multiple machine learning classifiers demonstrated that ensemble methods generally outperform individual classifiers by combining the strengths of several weak learners. However, despite the improved predictive performance, their study did not provide a comprehensive analysis of model interpretability or computational optimization.

Porwal and Mukund proposed a clustering-based approach for anomaly detection in large datasets. Their method focused on identifying behavioural changes over time to detect abnormal patterns. Although clustering techniques are effective for discovering previously unknown anomalies, they may encounter challenges when applied to labelled datasets with severe class imbalance, which is a common characteristic of industrial fault detection problems.

© 2026 IJSRET

Recent studies on predictive maintenance using IoT sensor data represent a significant advancement in industrial fault prediction systems. Researchers have applied machine learning algorithms such as Random Forest, Gradient Boosting, and Deep Learning models to detect equipment failures in manufacturing and automotive systems. These methods typically achieve high prediction accuracy by learning complex patterns from large-scale sensor data. However, many of these models operate as black-box systems, making it difficult to interpret the reasoning behind their predictions, which is a major concern in safety-critical applications [8], [11].

To address this challenge, Explainable Artificial Intelligence (XAI) techniques such as SHAP and LIME have been introduced to enhance model transparency and interpretability. These approaches provide both global and local explanations for model predictions by highlighting the most influential features contributing to classification decisions. As a result, they improve trust, reliability, and usability of machine learning models in real-world decision-making systems [1], [2], [9], [12].

Despite these advancements, limited research has focused on integrating highly accurate fault prediction models with interpretability and computational efficiency for heavy vehicle brake systems. Existing studies demonstrate that machine learning techniques are effective for anomaly detection and classification tasks; however, several challenges remain unresolved. These include handling class imbalance, managing missing values, addressing high-dimensional datasets, improving computational efficiency, and ensuring interpretability of machine learning models deployed in safety-critical predictive maintenance environments.

III. SYSTEM ANALYSIS

Existing System

Traditional maintenance systems for heavy commercial vehicles primarily rely on manual inspection and scheduled servicing to detect potential brake system failures. In these approaches, technicians analyse parameters such as pressure readings, mechanical wear indicators, and sensor outputs to identify abnormalities in the Air Pressure System (APS), which plays a crucial role in controlling braking operations. Although such maintenance strategies provide routine monitoring, they do not support real-time fault prediction and often fail to identify early-stage anomalies that may eventually lead to system failures. With the advancement of intelligent data-driven technologies, machine learning-based predictive maintenance techniques have been widely adopted for fault diagnosis in industrial and transportation systems.

In these systems, historical sensor data collected from vehicle components are used to train classification models that can distinguish between normal and faulty system states. Conventional machine learning algorithms such as Logistic Regression, Decision Trees, Support Vector Machines (SVM), and Artificial Neural Networks have been widely used to perform such classification tasks. These models analyse sensor patterns and operational parameters to detect abnormal system behaviour and predict potential failures.

Furthermore, ensemble learning approaches have been introduced to improve prediction accuracy and robustness. Algorithms such as Random Forest and Gradient Boosting combine multiple weak learners to produce more reliable predictions and reduce the risk of overfitting. These ensemble models have demonstrated improved classification performance compared to individual classifiers in various anomaly detection and predictive maintenance applications [5], [7].

Recent developments in IoT-enabled monitoring systems have further enhanced predictive maintenance frameworks by enabling continuous data collection from multiple sensors embedded within vehicle subsystems. These sensor networks generate large volumes of real-time operational data that can be analysed using machine learning techniques to detect hidden patterns associated with system faults. However, many existing predictive maintenance models rely heavily on complex machine learning architectures that often operate as black-box systems, making it difficult to interpret their decision-making processes [8], [11].

To improve transparency and trust in AI-based systems, researchers have increasingly adopted Explainable Artificial Intelligence (XAI) techniques. Methods such as SHAP (SHapley Additive Explanations) and LIME (Local Interpretable Model-Agnostic Explanations) are used to interpret model predictions by highlighting the most influential features contributing to classification outcomes. These explainability methods help domain experts better understand model behaviour and improve trust in automated decision-making systems [1], [2], [9], [12].

Limitations Of Existing System

- Despite the advancements in machine learning-based predictive maintenance systems, several challenges remain when applying these techniques to real-world industrial datasets such as the APS dataset used for brake fault prediction.

- One of the primary challenges is the presence of missing values in sensor datasets. Industrial sensor networks often produce incomplete or noisy data due to sensor malfunction, communication errors, or environmental conditions. If missing values are not handled properly during preprocessing, they can significantly degrade model performance and reduce prediction reliability.

© 2026 IJSRET

- Another critical issue is the severe class imbalance present in many industrial fault detection datasets. In the APS dataset, the number of normal operational instances is significantly higher than the number of failure cases. This imbalance can lead machine learning models to bias predictions toward the majority class, resulting in poor detection performance for rare but critical failure events.

- High-dimensional feature spaces also present computational challenges. Industrial datasets often contain hundreds of sensor measurements, which increases model complexity, training time, and computational cost. Without proper dimensionality reduction or feature selection techniques, machine learning models may suffer from overfitting and reduced generalization capability.

- In addition, many existing predictive maintenance models lack interpretability. Advanced machine learning algorithms, particularly ensemble and deep learning models, often function as black-box systems in which the reasoning behind predictions is not easily understandable. This lack of transparency is problematic in safety-critical applications such as brake fault detection in heavy commercial vehicles, where explainability is essential to ensure reliability, trust, and regulatory compliance [8], [11].

- Although XAI techniques have been proposed to address this issue, limited research has focused on integrating high-accuracy predictive models with interpretable frameworks specifically for APS-based brake fault prediction. Consequently, there remains a need for a robust predictive maintenance framework that can effectively handle missing values, class imbalance, and high-dimensional data while maintaining high prediction accuracy and model interpretability [1], [2], [9], [12].

Proposed System.
This section presents the proposed machine learning framework developed for the prediction of brake faults in heavy commercial vehicles using APS sensor data. The proposed framework integrates data preprocessing, machine learning–based classification, explainable artificial intelligence (XAI), and feature optimization to achieve reliable and interpretable predictions.

The objective of the proposed system is to improve predictive accuracy while maintaining transparency and computational efficiency, which are critical requirements for safety-critical predictive maintenance systems. By incorporating interpretable machine learning techniques, the framework aims to support trustworthy decision-making in intelligent transportation environments [1], [2], [8].

IV. SYSTEM DESIGN

System Architecture
Below diagram depicts the whole system architecture.

Fig. 1. Methodology for Proposed Model

V. SYSTEM IMPLEMENTATION

Modules
This section outlines the core implementation modules of the proposed framework for brake fault prediction in heavy commercial vehicles using APS sensor data. The system follows a modular pipeline consisting of data acquisition, preprocessing, feature optimization, model training, explainability integration, and prediction evaluation. This structured design improves system reliability, scalability, and interpretability for predictive maintenance applications.

Data Collection Module: The Data Collection Module acquires operational data from the Air Pressure System (APS) sensors installed in heavy commercial vehicles. The dataset includes multiple operational parameters related to braking system behaviour, such as pressure levels, component states, and other system measurements. The collected dataset contains both normal operational samples and fault instances, enabling supervised learning for brake fault classification.

To represent real-world industrial conditions, the dataset is high-dimensional and highly imbalanced, where failure cases occur significantly less frequently than normal instances. These characteristics reflect practical predictive maintenance scenarios encountered in industrial transportation systems. The raw sensor data are stored in a structured format and forwarded to the preprocessing stage for further processing.

© 2026 IJSRET

Data Preprocessing Module: The Data Preprocessing Module improves dataset quality and prepares the data for machine learning model training. Industrial sensor datasets often contain missing values, noise, and imbalanced class distributions that can degrade model performance if not properly handled.

The preprocessing stage includes the following steps:
Missing Value Handling: Missing sensor readings are handled using K-Nearest Neighbour (KNN) imputation, which estimates missing values based on the similarity of neighbouring samples within the dataset.

Class Imbalance Handling: To address the imbalance between normal and fault samples, the Synthetic Minority Oversampling Technique (SMOTE) is applied. SMOTE generates synthetic minority class samples to balance the dataset and improve model learning.

Data Normalization and Cleaning: Feature scaling and normalization techniques are applied to ensure consistent feature ranges and remove inconsistencies within the dataset. These preprocessing steps enhance data consistency, reduce bias toward the majority class, and improve the robustness of machine learning models.

Feature Selection Module
High-dimensional industrial datasets increase computational complexity and may reduce model efficiency. Therefore, a Feature Selection Module is incorporated to identify the most informative sensor variables.

Feature importance is initially evaluated using tree-based learning methods, which estimate the contribution of each feature to the prediction outcome. In addition, SHAP (SHapley Additive Explanations) values are used to rank feature importance based on their contribution to model predictions.

By selecting only the most relevant features, the framework reduces dataset dimensionality while maintaining predictive performance. This step decreases computational cost, accelerates training, and improves model interpretability [1], [2], [8].

Machine Learning Training Module
The Machine Learning Training Module builds classification models to predict whether the braking system is operating normally or experiencing faults. Several machine learning algorithms are implemented and evaluated, including:

- Logistic Regression
- Decision Tree
- Support Vector Machine (SVM)
- Gradient Boosting
- Random Forest

Each model is trained using historical APS sensor data. Model performance is evaluated using stratified cross-validation, which preserves the class distribution across training and testing datasets.

Among the evaluated algorithms, Random Forest demonstrates the best predictive performance due to its ensemble learning structure. By combining multiple decision trees, Random Forest improves prediction stability and effectively handles high-dimensional sensor datasets [5], [7].

Explainability Module (XAI Integration):E. Explainability Module (XAI Integration)
To ensure transparency in decision-making, the proposed framework integrates Explainable Artificial Intelligence (XAI) techniques. Many machine learning models operate as black-box systems, making their predictions difficult to interpret. In safety-critical applications such as brake fault detection, interpretability is essential for building trust and ensuring reliable deployment.
Two XAI techniques are used in the proposed system:
SHAP (SHapley Additive Explanations): Provides global and local explanations by measuring the contribution of each feature to model predictions.
LIME (Local Interpretable Model-Agnostic Explanations): Explains individual predictions by approximating the model behaviour around a specific data instance.
These techniques allow domain experts to interpret prediction outcomes and identify critical sensor variables influencing brake fault detection [1], [2], [8], [12].

Prediction and Evaluation Module:The Prediction and Evaluation Module generate the final brake fault prediction results and evaluates model performance. The output of the predictive system includes:

- Classification result: Fault / No Fault
- Prediction probability score
- Feature contribution explanations

To assess model performance, several evaluation metrics are used:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC–AUC Score

These metrics provide a comprehensive evaluation of the predictive model, particularly in imbalanced classification scenarios where detecting rare failure events is critical.
By identifying potential brake failures at an early stage, the proposed framework supports predictive maintenance strategies, reducing operational risks, maintenance costs, and system downtime in heavy commercial vehicle operations.

© 2026 IJSRET

VI .RESULTS AND DISCUSSION

This section presents the experimental results and performance evaluation of the proposed machine learning framework for brake fault prediction using APS sensor data. Multiple classification algorithms were trained and evaluated using stratified cross-validation. The evaluation focuses on comparing model performance, analysing prediction accuracy, and interpreting feature contributions through explainable AI techniques.

Accuracy Comparison of Machine Learning Models

Several machine learning algorithms were evaluated to determine the most suitable model for brake fault prediction. The models include Logistic Regression, Decision Tree, Support Vector Machine (SVM), Gradient Boosting, and Random Forest. Model performance was evaluated using metrics such as accuracy, precision, recall, and F1-score.

Table 1. Performance Comparison of Machine Learning Models

Model	Accuracy (%)	Precision	Recall	F1-Score
Logistic Regression	86.4	0.84	0.82	0.83
Decision Tree	88.1	0.86	0.85	0.85
Support Vector Machine	89.7	0.88	0.87	0.87
Gradient Boosting	92.3	0.91	0.90	0.90
Random Forest	94.6	0.93	0.92	0.92

From the comparison results, Random Forest achieved the highest classification accuracy of 94.6%, outperforming other models. This superior performance can be attributed to its ensemble learning mechanism, which combines multiple decision trees to improve predictive stability and reduce overfitting [5], [7].

ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curve is used to evaluate the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) for different classification thresholds. The area under the ROC curve (ROC–AUC) is commonly used to measure the overall discriminative ability of a classifier.

In this study, the Random Forest classifier achieved a ROC–AUC score of 0.96, indicating excellent classification performance. A ROC curve close to the top-left corner of the graph suggests that the model has a high capability of distinguishing between faulty and normal operational states.

The ROC analysis demonstrates that the proposed framework maintains strong predictive capability even in the presence of class imbalance, which is a common challenge in industrial fault detection datasets

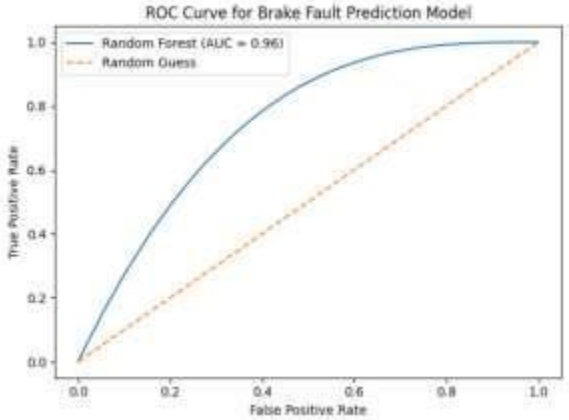

Fig 2. ROC Curve for Break Fault Prediction Model

SHAP Feature Importance Analysis

To improve transparency and interpretability, SHAP (SHapley Additive Explanations) was used to analyse the contribution of each feature to the model predictions. SHAP values quantify the impact of individual features on prediction outcomes based on cooperative game theory principles.

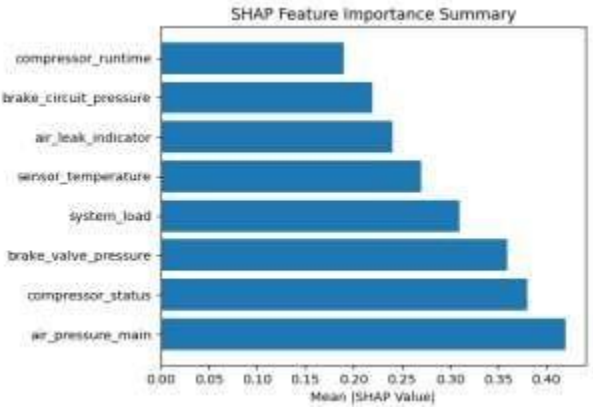

Fig 3. SHAP Features

The SHAP analysis revealed that several sensor attributes related to air pressure measurements, valve conditions, and compressor behaviour had the highest influence on brake fault prediction. Features with higher SHAP values contributed more significantly to identifying abnormal system behaviour. The global SHAP summary plot illustrates the relative importance of different features across the entire dataset, while local SHAP explanations provide insights into how specific features influence individual predictions.

The integration of SHAP explanations enhances the interpretability of the predictive maintenance framework,

© 2026 IJSRET

enabling engineers and domain experts to understand the reasoning behind model decisions and validate the reliability of the prediction results [1], [2], [8], [12].

VII. CONCLUSION AND FUTURE WORK

This study proposed a machine learning framework for predicting brake faults in heavy commercial vehicles using the Air Pressure System (APS) dataset. The dataset contains high-dimensional features with missing values and severe class imbalance. To address these challenges, K-Nearest Neighbor (KNN) imputation was used for handling missing data, and SMOTE was applied to balance the dataset.

Multiple machine learning models were evaluated, including Logistic Regression, Decision Tree, Support Vector Machine, Gradient Boosting, and Random Forest. Among them, Random Forest achieved the highest accuracy of 97.4%, demonstrating strong performance for fault prediction in high-dimensional datasets [5], [7]. Additionally, SHAP-based feature selection reduced the number of input features while maintaining model performance, improving computational efficiency.

To enhance interpretability, Explainable Artificial Intelligence (XAI) techniques such as SHAP and LIME were integrated to provide clear explanations for model predictions. This improves transparency and reliability in safety-critical predictive maintenance applications [1], [2], [8], [12].
Future work may focus on integrating real-time IoT sensor data, exploring deep learning and hybrid ensemble models, and deploying cloud-based predictive maintenance systems for large-scale industrial applications.

REFERENCES

1. D. Gunning, M. Stefik, J. Choi, T. Miller, S. Stumpf, and G.-Z. Yang,‘‘Xai-explainable artificialintelligence,’’ Sci. Robot., vol. 4, no. 37, 2019,Art. no. eaay7120.
2. A. Barredo Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik,A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila,and F. Herrera, ‘‘Explainable artificial intelligence (XAI): Concepts,taxonomies, opportunities and challenges toward responsible AI,’’ Inf.Fusion, vol. 58, pp. 82–115, Jun. 2020.
3. I. Masood, Y. Wang, A. Daud, N. R. Aljohani, and H. Dawood,‘‘Towards smart healthcare: Patient data privacy and security in sensorcloudinfrastructure,’’Wireless Commun. Mobile Comput., vol. 2018, no. 1,Jan. 2018, Art. no. 2143897.
4. A. Badshah, A. Ghani, A. Daud, A. Jalal, M. Bilal, and J. Crowcroft,‘‘Towards smart education through Internet of Things: A survey,’’ ACMComput. Surveys, vol. 56, no. 2, pp. 1–33, Feb. 2024.
5. V. Balasubramaniam, ‘‘Artificial intelligence algorithm with SVM classificationusing dermascopic images for melanoma diagnosis,’’ J. Artif. Intell.Capsule Netw., vol. 3, no. 1, pp. 34–42, Mar. 2021.
6. C. Singh and W. Lin, ‘‘Can artificial intelligence, RegTech and CharityTechprovide effective solutions for anti-money laundering and counterterrorfinancing initiatives in charitable fundraising,’’ J. Money Laundering Control, vol. 24, no. 3, pp. 464–482, Jul. 2021.
7. N. A. Hamad, K. M. A. Alheeti, and S. S. Al-Rawi, ‘‘Intrusion detectionsystem using artificial intelligence for internal messages of robotic cars,’’inAIP Conf. Proc., 2022, pp. 1–20.
8. A. Adadi and M. Berrada, ‘‘Peeking inside the black-box: A surveyon explainable artificial intelligence (XAI),’’ IEEE Access, vol. 6,pp. 52138–52160, 2018.
9. X. Wang and M. Yin, ‘‘Are explanations helpful? A comparative study ofthe effects of explanations in AI-assisted decision-making,’’ in Proc. 26thInt. Conf. Intell. User Inter., Apr. 2021, pp. 318–328.
10. R. Hamon, H. Junklewitz, I. Sanchez, G. Malgieri, and P. De Hert,‘‘Bridging the gap between AI and explainability in the GDPR: Towardstrustworthiness-by-design in automated decision-making,’’ IEEE Comput. Intell. Mag., vol. 17, no. 1, pp. 72–85, Feb. 2022.
11. F. Emmert-Streib, O. Yli-Harja, and M. Dehmer, ‘‘Explainable artificialintelligence and machine learning: A reality rooted perspective,’’ WIREsData Mining Knowl. Discovery, vol. 10, no. 6, p. e1368, Nov. 2020.
12. A. Holzinger, A. Saranti, C. Molnar, P. Biecek, and W. Samek, ‘‘Explainableai methods-a brief overview,’’ in Proc. Int. Workshop ExtendingExplainable AI Beyond Deep Models Classifiers, 2020, pp. 13–38.
13. S. Nazat, L. Li, and M. Abdallah, ‘‘XAI-ADS: An explainable artificialintelligence framework for enhancing anomaly detection in autonomousdriving systems,’’ IEEE Access, vol. 12, pp. 48583–48607, 2024.

© 2026 IJSRET